INTELIGENȚĂ ARTIFICIALĂ

Sisteme inteligente

Sisteme care învaţă singure

– generative AI–

Laura Dioşan

Sumar

A. Scurtă introducere în Inteligenţa Artificială (IA)

C. Sisteme inteligente
- Sisteme care învaţă singure
  - Arbori de decizie
  - **Reţele neuronale artificiale**
  - kNN
  - Algoritmi evolutivi
  - Maşini cu suport vectorial
- Sisteme bazate pe reguli
- Sisteme hibride

B. Rezolvarea problemelor prin căutare
- Definirea problemelor de căutare
- Strategii de căutare
  - Strategii de căutare neinformate
  - Strategii de căutare informate
  - Strategii de căutare locale (Hill Climbing, Simulated Annealing, Tabu Search, Algoritmi evolutivi, PSO, ACO)
  - Strategii de căutare adversială

Materiale utile

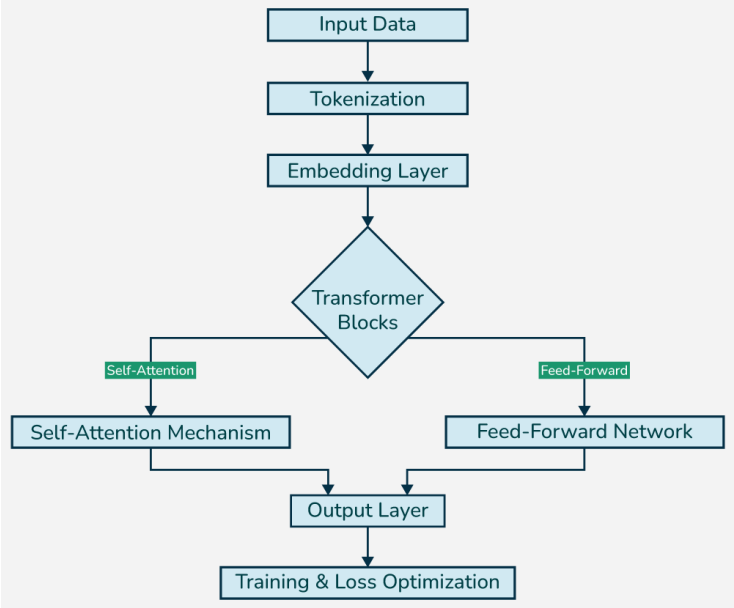
- https://www.tensorflow.org/text/tutorials/word2vec
- *https://radimrehurek.com/gensim/models/word2vec.html*
- *https://www.ruder.io/word-embeddings-1/*
- *https://aylien.com/blog/a-review-of-the-recent-history-of-natural-language-processing?ref=ruder.io*
- *https://nlp.stanford.edu/~manning/ - a lot of lectures about NLP*

Rețele neuronale artificiale

- Rețele neuronale dense (fully-connected)
- Rețele neuronale convolutive
- Transformers

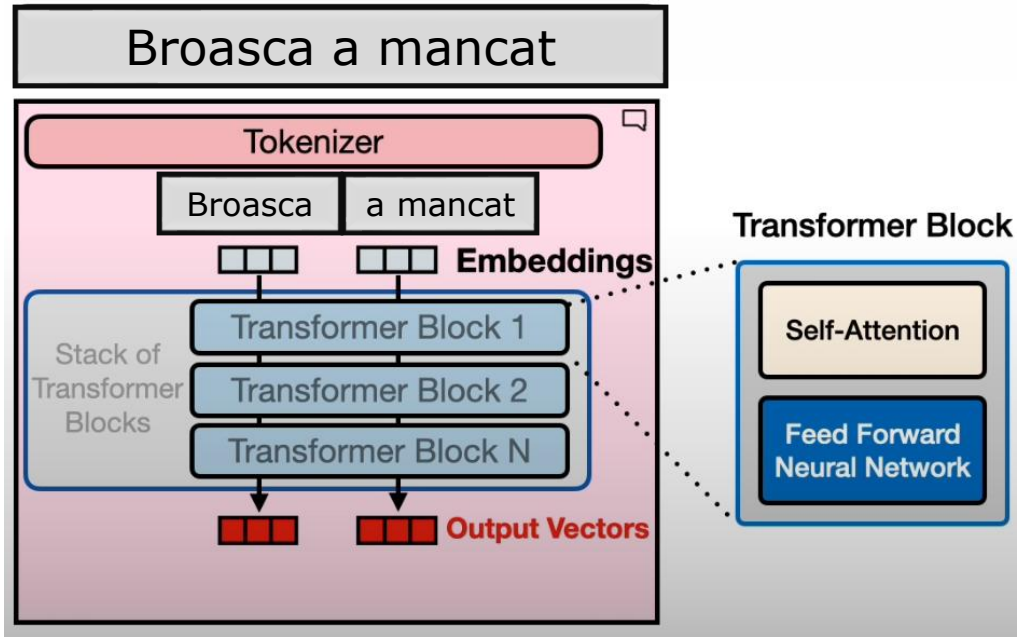
Procesarea textelor/limbajelor

- Large language Models (LLMs)
  - Tokenisation
  - Embeddings
  - Transformer & Self-attention mechanism
  - Feed-forward network



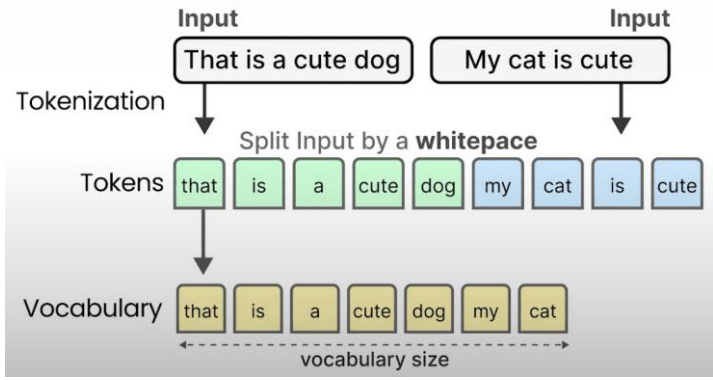
Procesarea textelor/limbajelor

- Large language Models (LLMs)
  - **Tokenisation**
  - Embeddings
  - Transformer & Self-attention mechanism
  - Feed-forward network



Large language Models (LLMs)

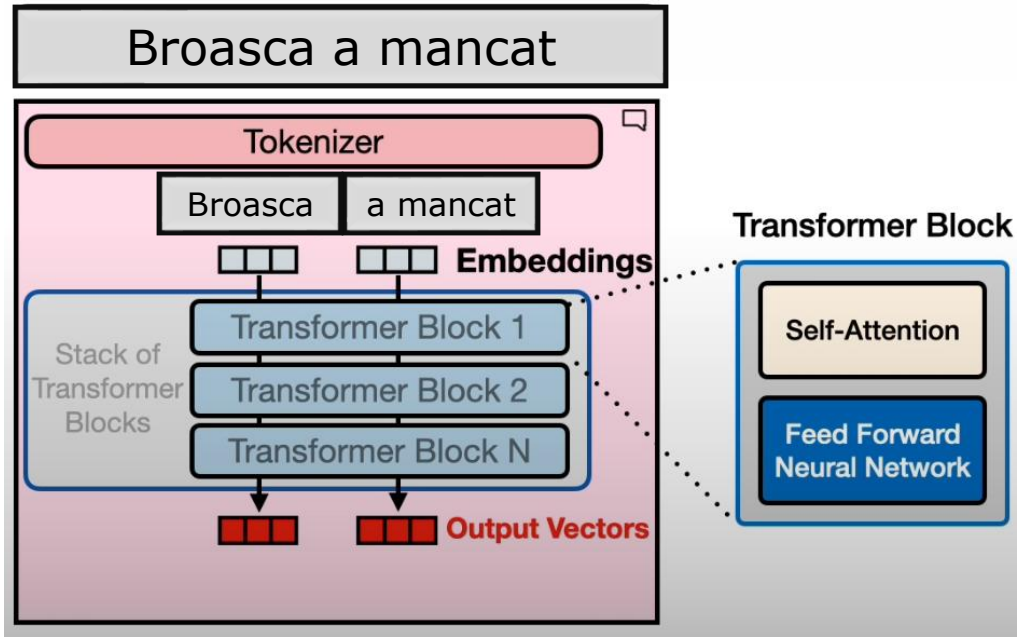
- Tokenisation



  - Token-based models
    - White Space: Splits on spaces (simple but limited)
    - Word: Breaks into words (common for English)
    - Sentence: Divides text into sentences
    - Character: Splits into individual characters
    - N-gram: Creates sequences of n items/elements
      - fastText (2016)
    - subword: Breaks words into smaller parts
      - Byte Pair Encoding (2016): Merges common character pairs → GPT, GPT2, GPT3
      - WordPiece (2012): Google's method for balancing words and subwords → BERT
      - Unigram (2018) → T5, AIBERT
  - Token-free models
    - CharFormer
    - ByT5
    - MegaByte (2023)
    - Byte-latent transformer (2024)
  - Useful resources
    - tokenizers.ipynb
    - Karpathy's tutorial https://www.youtube.com/watch?v=zduSFxRajkE
    - Hugging Face tutorial https://huggingface.co/learn/llm-course/en/chapter2/4

Procesarea textelor/limbajelor

- Large language Models (LLMs)
  - Tokenisation
  - **Embeddings**
  - Transformer & Self-attention mechanism
  - Feed-forward network



LLMs – word embeddings

- De ce reprezentari vectoriale ale intelesului unui cuvant/text?
  - Permit determinarea similaritatii intre cuvinte/texte
    - ***fast*** is similar to ***rapid***
    - ***tall*** is similar to ***height***

Question answering:

➢*Q: "How **tall** is Mt. Everest?"*
*Candidate A: "The official **height** of Mount Everest is 29029 feet"*

LLMs – word embeddings

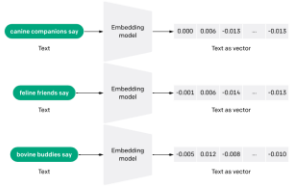
- Intuitia din spatele similaritatii
  - Exemplu: Ce este o Sirrus X 3.0?

*The **Sirrus X 3.0** invites you to explore beyond boundaries. With confidence-inspiring tires, an upright riding position, and intuitive components, **Sirrus X 3.0** is your ticket to adventure.*

  - Din context, o persoana poate ghici ca Sirrus X 3.0 este un model de bicicleta

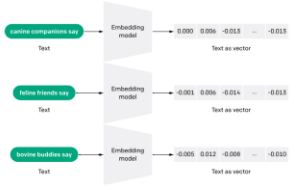
  - Pentru un algoritm, intuitia e ca **doua cuvinte sunt similar daca ele sunt folosite in context similare**

Large language Models (LLMs)



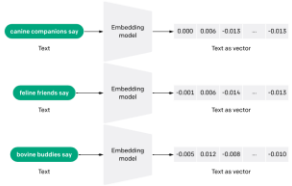
- Word Embeddings
  - Frequency-based embeddings
    - One-hot encoding
    - TF-IDF
    - Co-occurrence matrix
  - Prediction based embeddings → Word / subword embeddings (learnt embeddings)
    - De ce?
      - Token ➔ reprezentari dense
    - Cum?
      - Word2vec (Skip-gram or Continuous Bag of Words (CBOW), FastText, GloVe, altele

Large language Models (LLMs)



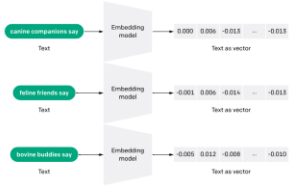
- Word Embeddings
  - Vectori rari *vs*. vectori densi
    - Vectorii clasici → matricea de aparitie/co-ocurenta a termenilor
      - lungi (length |V|= 20,000 -> 50,000)
      - rari (f multe elemente sunt 0)
    - Alternativa: vectori invatati (prin AI/ML)
      - scurti (length 200-1000)
      - densi (multe elemente nu sunt 0)
    - De ce vectori densi?
      - Vectorii scurti -> folositi mai usor ca si features in algoritmii de invatare (mai putini coeficienti de invatat)
      - Vectorii densi pot generaliza mai bine, captand sinonimia termenilor
        - Bike – scooter
        - Car – automobile
        - House – apartment

Large language Models (LLMs)

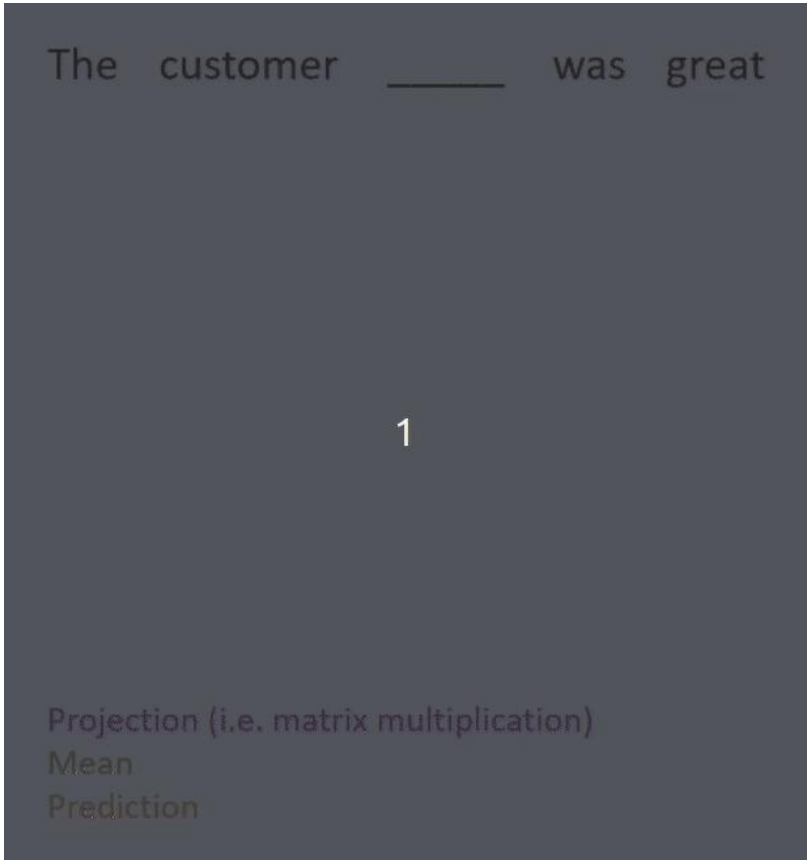


- Word Embeddings
  - Learnt embeddings
    - Static (context-free)

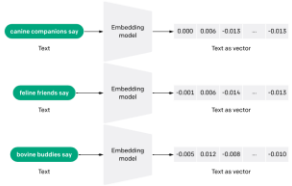
Large language Models (LLMs)



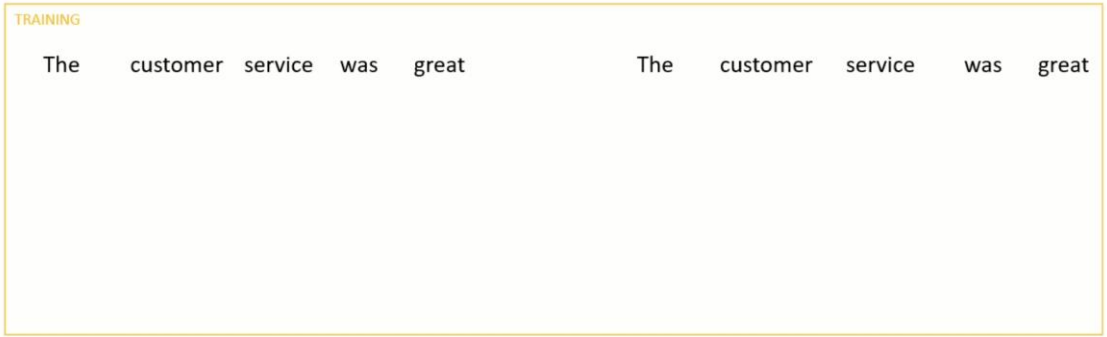
- Word Embeddings
  - Learnt embeddings
    - Static (context-free)
      - Word2vec (2013)
      - GLoVe (2014)



Large language Models (LLMs)

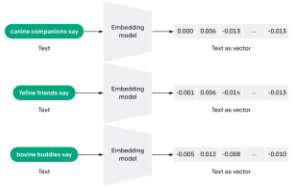


- Word Embeddings
  - Learnt embeddings
    - Static (context-free)
      - Word2vec (2013)
      - GLoVe (2014)
    - Dinamic (context-based)
      - ELMo (2018)



Forward LSTM
Backward LSTM
Prediction
Concatenation
Weighted average

Large language Models (LLMs)

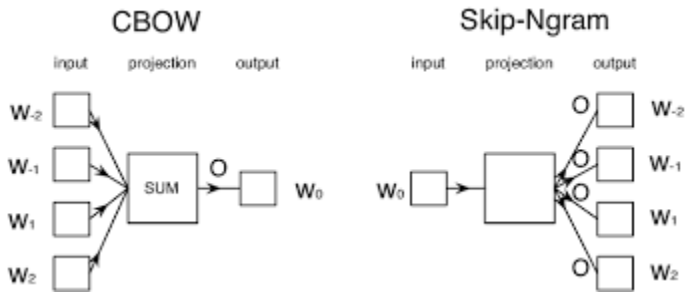


- Word Embeddings
  - Learnt embeddings
    - Static (context-free)
      - Word2vec (2013)
      - GLoVe (2014)
    - Dinamic (context-based)
      - ELMo (2018)
      - BERT (2019)

Large language Models (LLMs) – Word Embeddings
Modele de predictie (invatare) a reprezentarilor

Modelul Word2vec

- Versiuni
  - **Skip-gram** (Mikolov et al. 2013a),
  - **CBOW** (Mikolov et al. 2013b)



- Se invata reprezentari, numite embeddings, ca parte din procesul de predictive/generare a textului
- Se antreneaza o retea neuronala pentru prezicerea urmatorului cuvant
- Avantaje
  - Rapid, simplu de antrenat
  - Modele gata antrente disponibile online

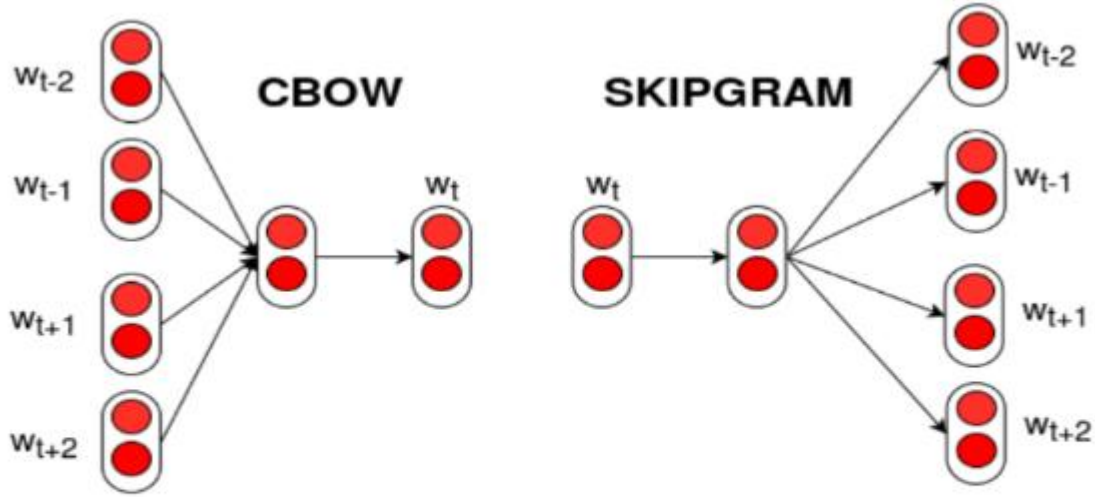
Large language Models (LLMs) – Word Embeddings
Modele de predictie (invatare) a reprezentarilor

- Invatare supervizata fara etichetare manuala de date
  - Text: "*She rides a bike. He rides a scooter in park. My father drives a motorcycle. …*"
  - Vocabular V = {*she, ride, bike, he, scooter, park, my, father, drive, motorcycle*}
  - Perechi (context, cuvant)

Negative sampling

[she ride] bike	1		[she ride] he	0
[ride bike] he	1		[she ride] scooter	0
[he ride] scooter	1		[she ride] park	0
[ride scooter] park	1		[she ride] my	0
[my father] drive	1		[she ride] father	0
[father drive] motorcycle	1		[she ride] drive	0
			[ride bike] scooter	0
			[ride bike] park	0
			…	
			[father drive] she	0

Large language Models (LLMs) – Word Embeddings - Arhitecturi



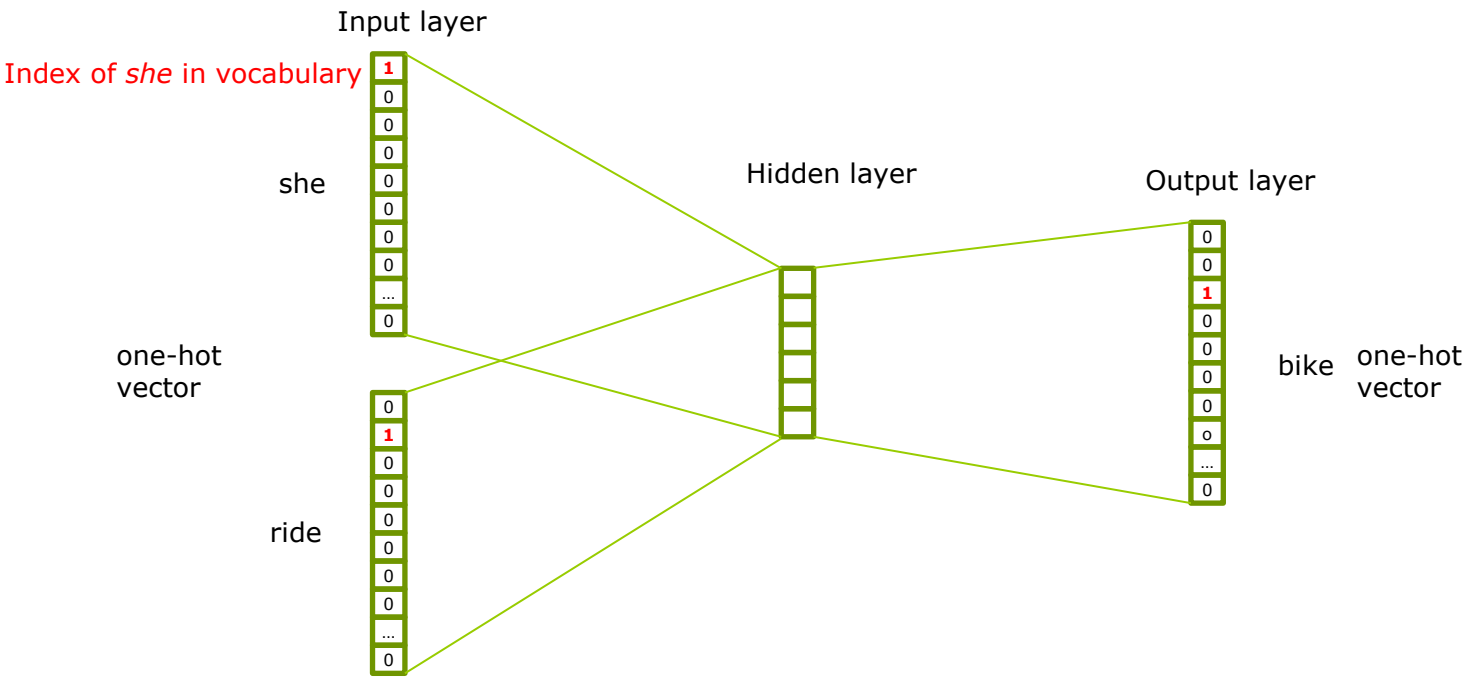
$$\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\leq j\leq c, j\neq 0} logp(w_t|w_{t+j})$$

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{-c\leq j\leq c, j\neq 0} logp(w_{t+j}|w_t)$$

Large language Models (LLMs) – Word Embeddings - Arhitecturi

CBOW (Continuous Bag of Words)

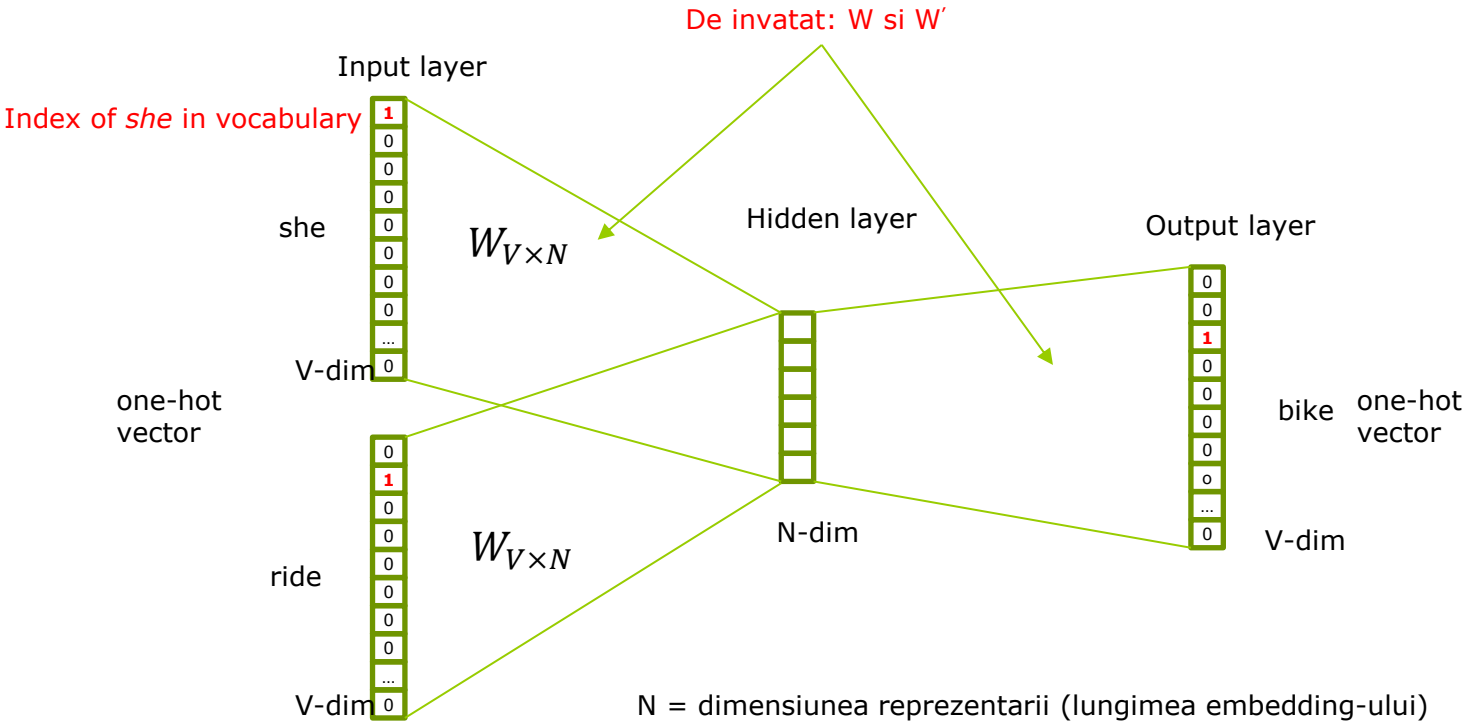
	she	ride	bike	he	scooter	park	my	father	drive	motorcycle
She	1	0	0	0	0	0	0	0	0	0
Ride	0	1	0	0	0	0	0	0	0	0
Bike	0	0	1	0	0	0	0	0	0	0



Large language Models (LLMs) – Word Embeddings - Arhitecturi

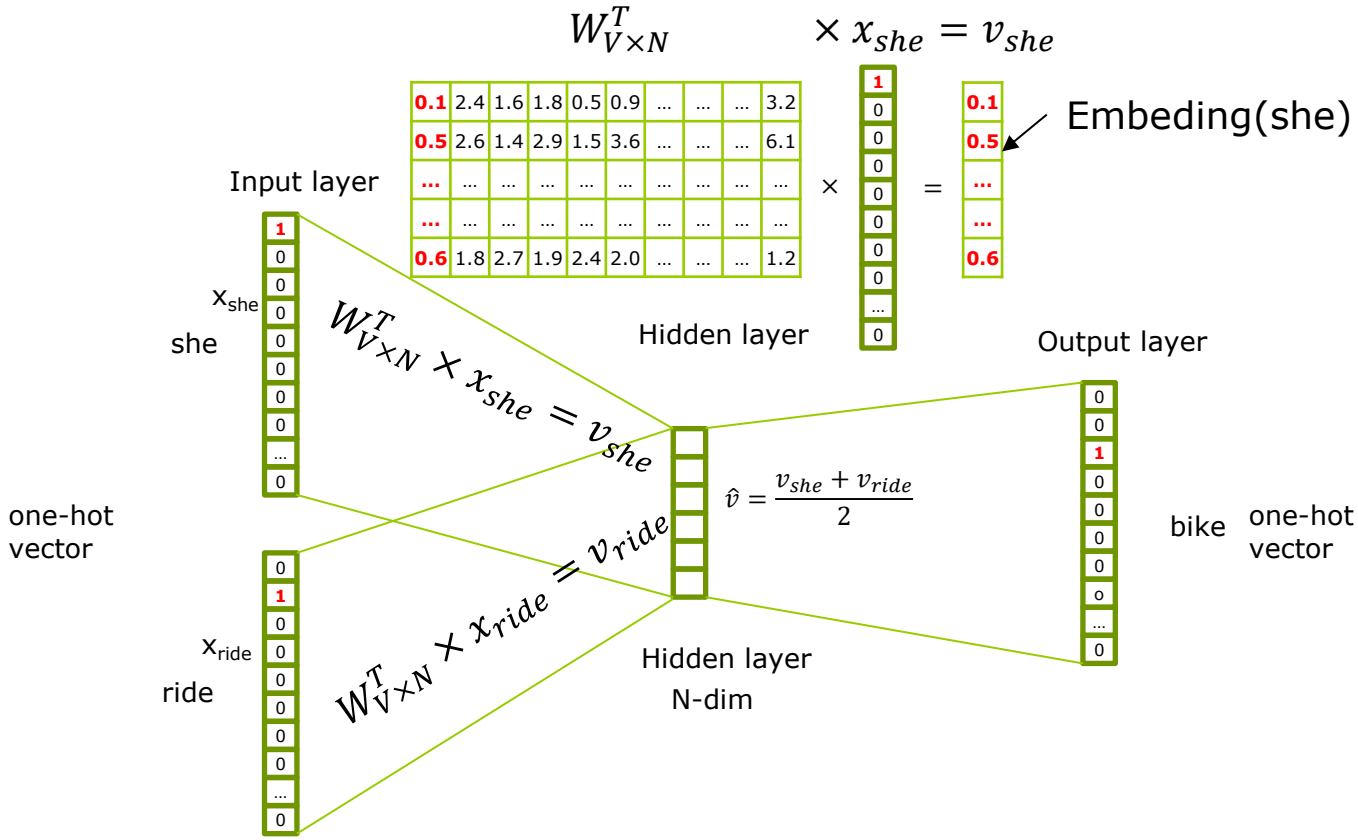
CBOW (Continuous Bag of Words)

	she	ride	bike	he	scooter	park	my	father	drive	motorcycle
She	1	0	0	0	0	0	0	0	0	0
Ride	0	1	0	0	0	0	0	0	0	0
Bike	0	0	1	0	0	0	0	0	0	0



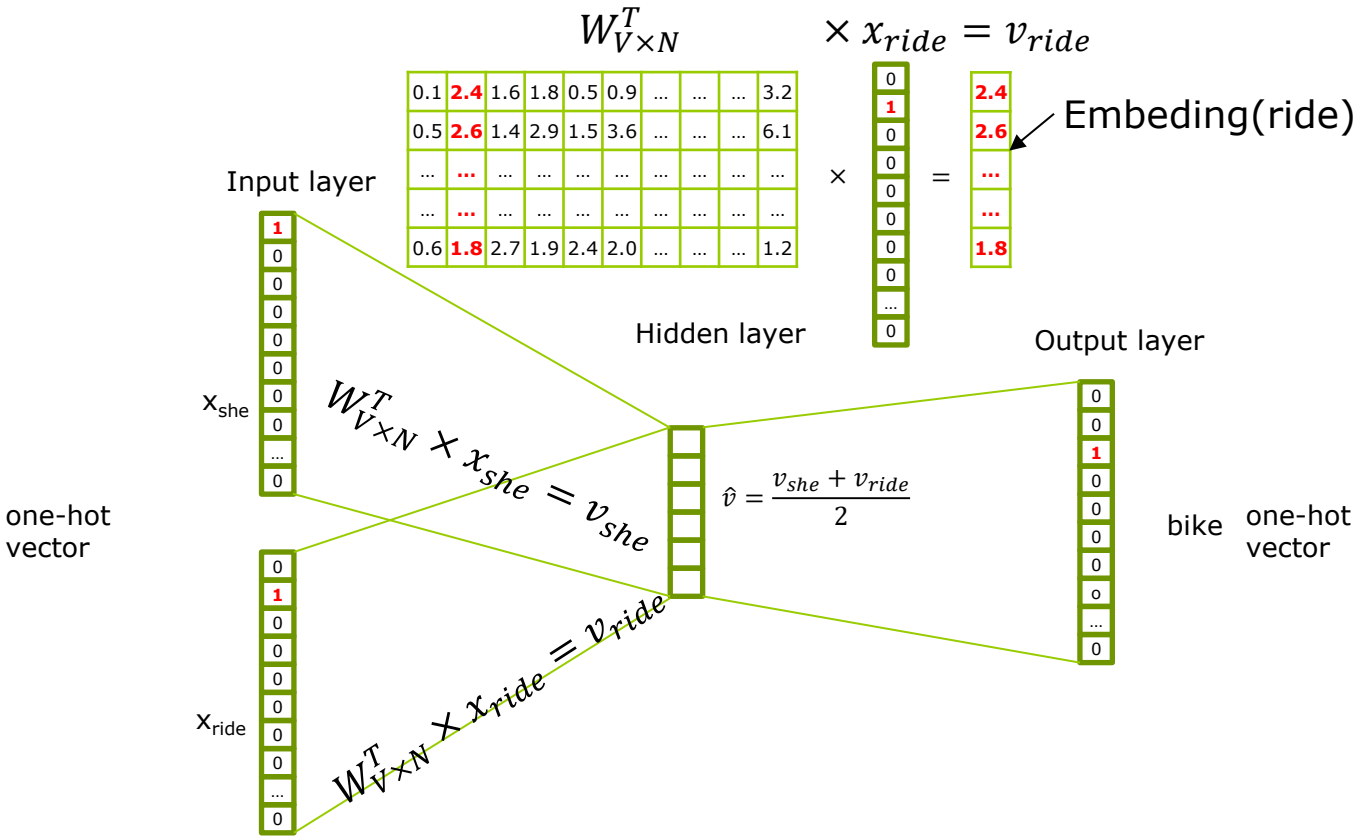
Large language Models (LLMs) – Word Embeddings - Arhitecturi

CBOW (Continuous Bag of Words)



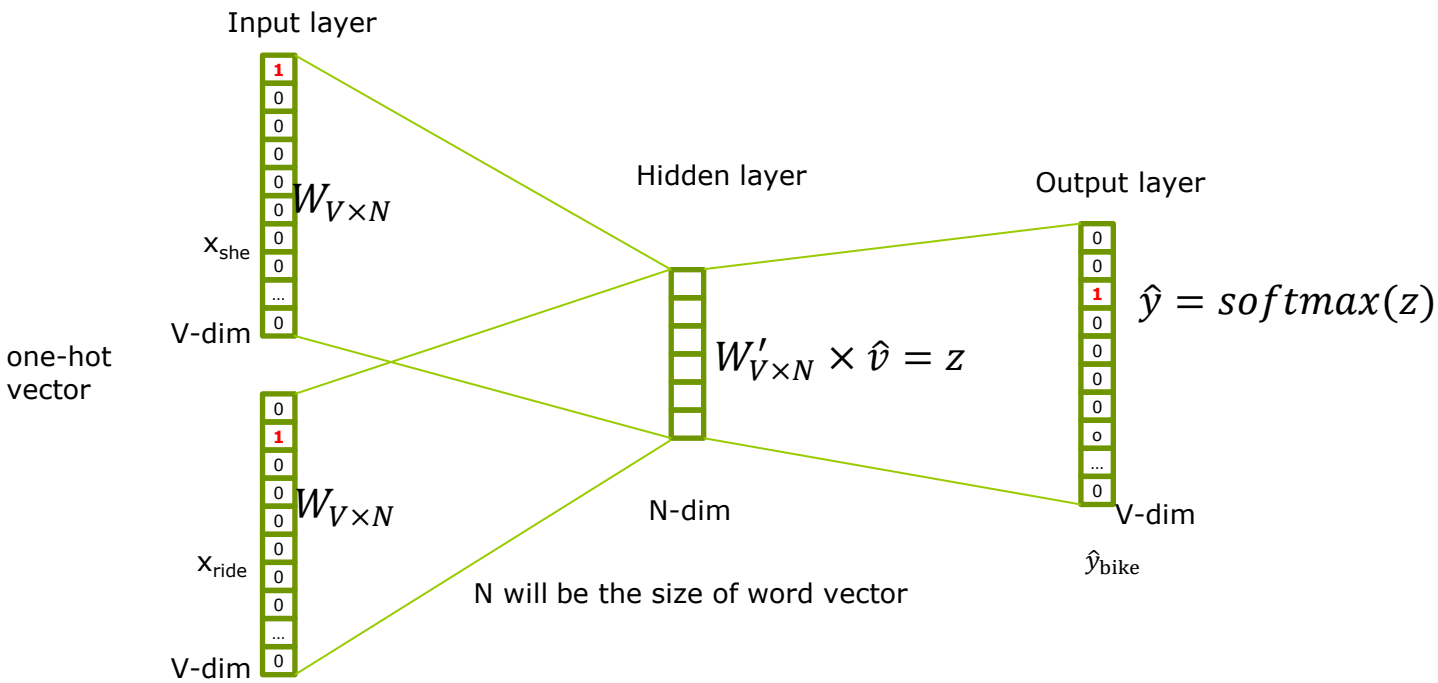
Large language Models (LLMs) – Word Embeddings - Arhitecturi

CBOW (Continuous Bag of Words)



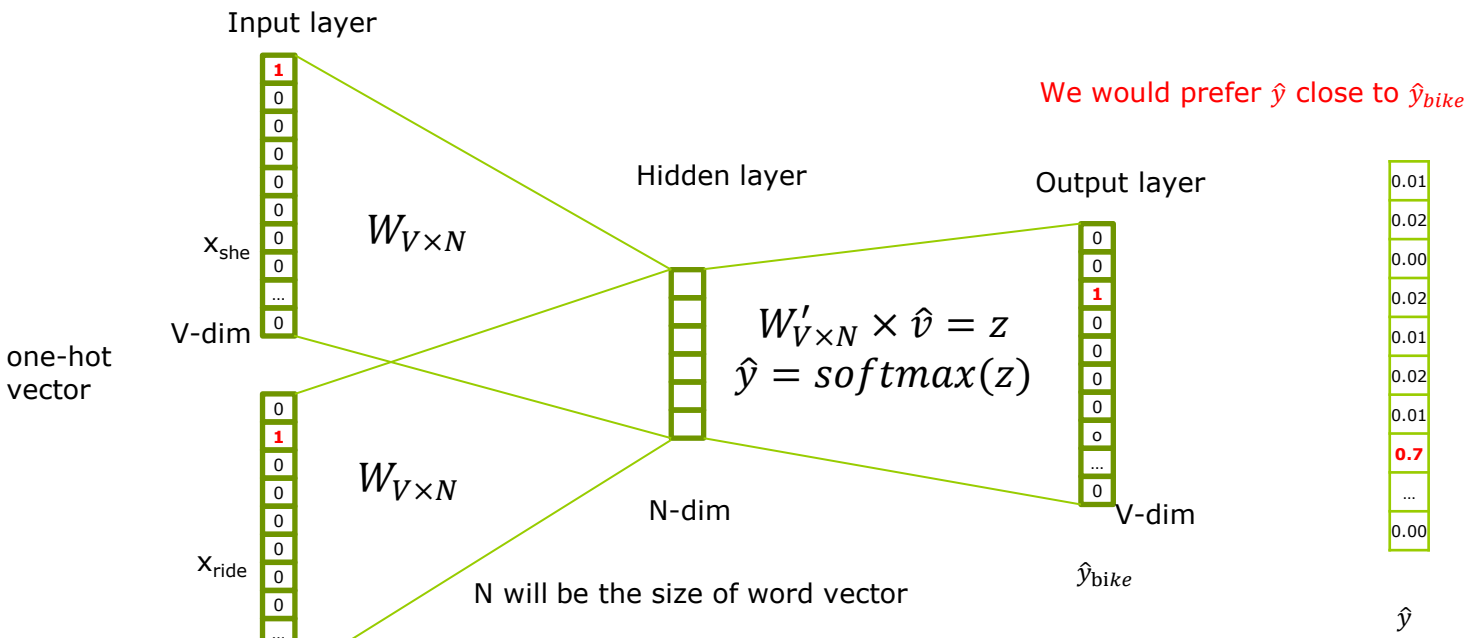
Large language Models (LLMs) – Word Embeddings - Arhitecturi

CBOW (Continuous Bag of Words)

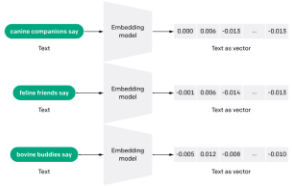


Large language Models (LLMs) – Word Embeddings - Arhitecturi

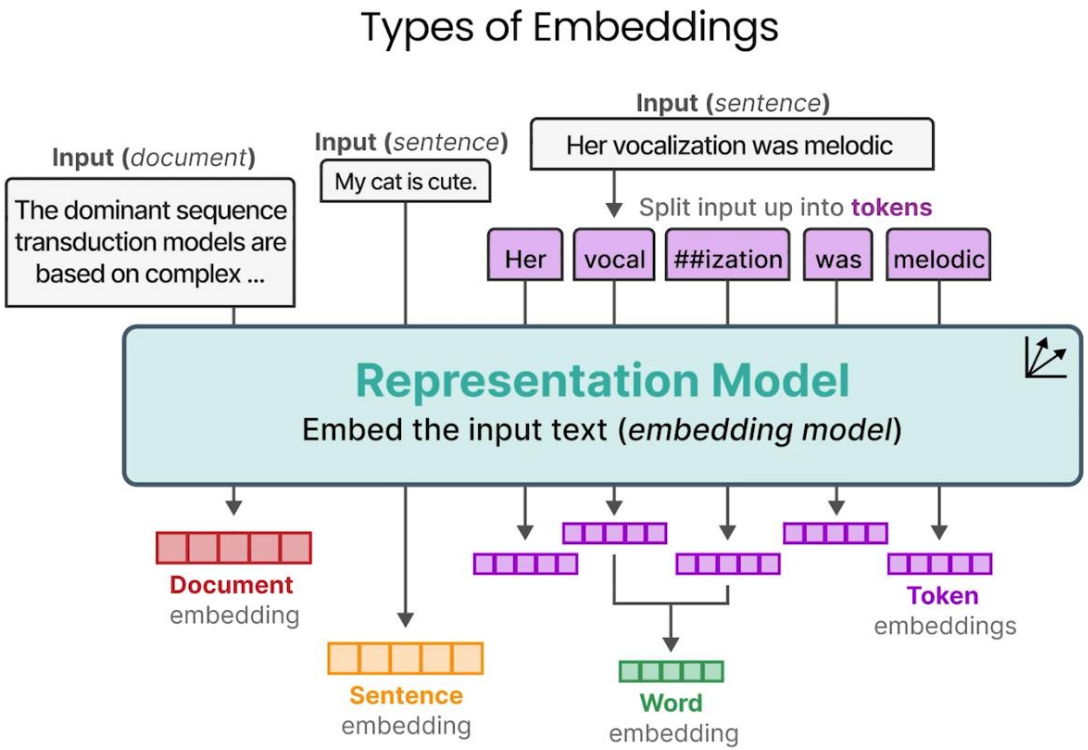
CBOW (Continuous Bag of Words)



Large language Models (LLMs)

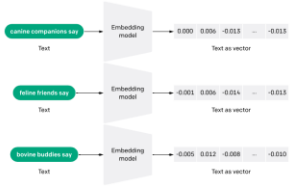


- Word Embeddings



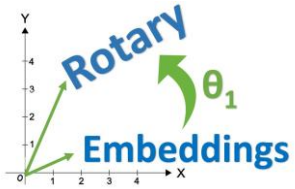
- Great resources
  - Word Embeddings https://lena-voita.github.io/nlp_course/word_embeddings.html
  - The Illustrated Word2vec – Jay Alammar – Visualizing machine learning one concept at a time https://jalammar.github.io/illustrated-word2vec/

Large language Models (LLMs)



- Word Embeddings
  - Frequency-based embeddings
  - Prediction based embeddings → Word / subword embeddings (learnt embeddings)
  - **Positional embeddings**

Large language Models (LLMs)



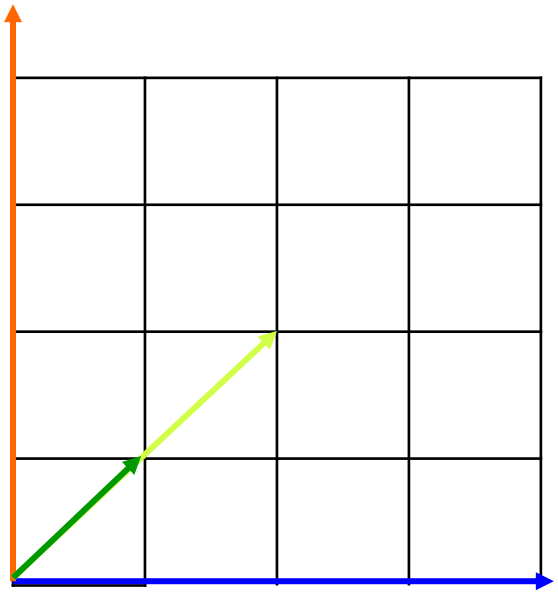
- Word Embeddings
  - Positional embeddings
    - Ce valori conține PE?
      - Principii
        - Embedding-urile trebuie sa păstreze distanța originală între cuvinte
          - dist(Emb(câine),Emb(pisică)) < dist(Emb(câine),Emb(fereastră))
          - Dist(Emb($word_k$),Emb($word_{k+1}$))=Dist(Emb($word_p$),Emb($word_{p+1}$))
        - Ortogonalitate - Embedding-urile cuvintelor ne-conectate să fie perpendiculare
          - Emb(ușă) $\perp$ Emb(pește)
          - Similaritate (ușă, pește) = 0
        - Embedding-uri independente de lungimea propoziției
          - $\rightarrow$ putere de generalizare (train *vs*. test)
          - $\rightarrow$ *bounded*
      - Reprezentări / valori posibile
        - Poziția efectivă (1,2,3,4)
        - Poziția în reprezentare 1-hot encoding
        - Poziția în reprezentare binară
        - Poziția în reprezentare polară

Pozitia in propozitie

- Exemplu
  - O propoziție cu 5 cuvinte și d = 1
  - PE(cuv) = poziția cuvântului

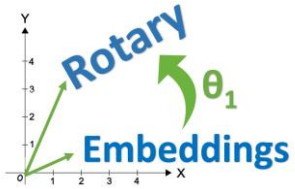
0
1
2
3
4

Dist	w0	w1	w2	w3	w4
w0	0	1	2	3	4
w1	1	0	1	2	3
w2	2	1	0	1	2
w3	3	2	1	0	1
w4	4	3	2	1	0



sim(PE(wi), PE(wj))!=0, orice i≠j

Pozitia normalizată in propozitie



- Exemplu
  - O propoziție cu 5 cuvinte și d = 1
  - PE(cuv) = poziția cuvântului

0/5
1/5
2/5
3/5
4/5

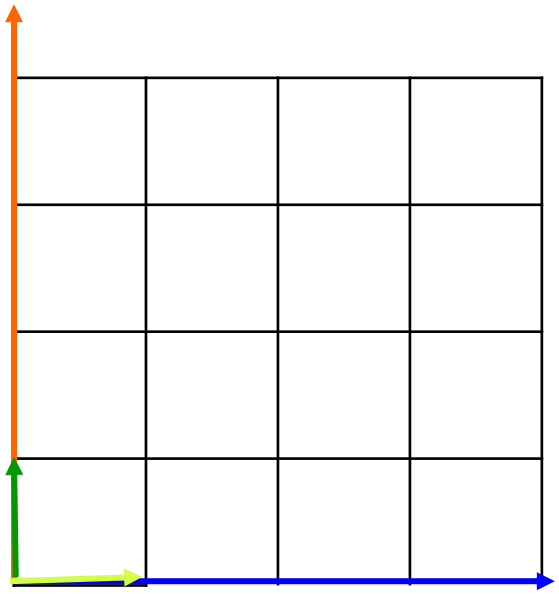
  - Problema: al k-lea cuvânt într-o propozției de N cuvinte are alt embedding decât al k-lea cuvânt într-o propoziție cu M cuvinte

1-hot encoding

- Exemplu
  - O propoziție cu 5 cuvinte și d = 6
  - PE(cuv) = reprezentarea 1-hot a poziției cuvântului

0	0	0	0	0	1
0	0	0	0	1	0
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	0	0	0

Dist	w0	w1	w2	w3	w4
w0	0	1.4	1.4	1.4	1.4
w1	1.4	0	1.4	1.4	1.4
w2	1.4	1.4	0	1.4	1.4
w3	1.4	1.4	1.4	0	1.4
w4	1.4	1.4	1.4	1.4	0



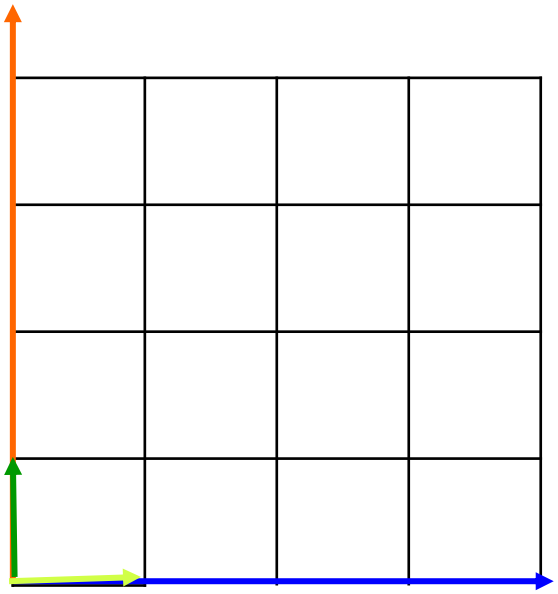
sim(PE(wi), PE(wj))=1.4, orice i≠j

Binary encoding

- Exemplu
  - O propoziție cu 5 cuvinte și d = 6
  - PE(cuv) = reprezentarea binară a poziției cuvântului

0	0	0	0	0	0
0	0	0	0	0	1
0	0	0	0	1	0
0	0	0	0	1	1
0	0	0	1	0	0

Dist	w0	w1	w2	w3	w4
w0	**0**	**1**	**1**	√2	**1**
w1	**1**	**0**	√2	**1**	√2
w2	**1**	√2	**0**	**1**	√2
w3	√2	**1**	**1**	0	√3
w4	**1**	√2	√2	√3	**0**

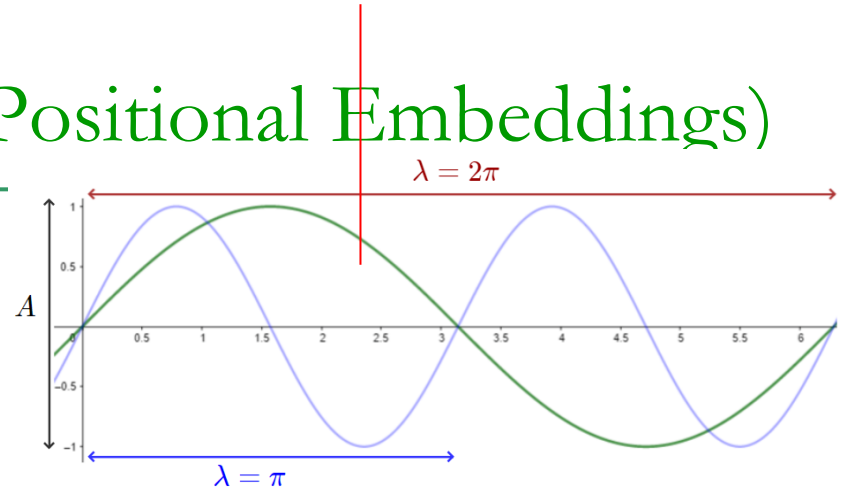


exista i, j a.î. sim(PE(wi), PE(wj))=0

  - Problema: dist nu e functie monotona!

Sin-based encoding (Rotary Positional Embeddings)

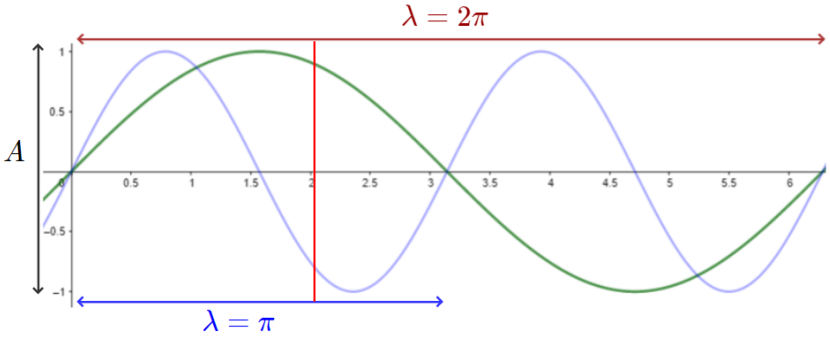
- Exemplu



- O propoziție cu 5 cuvinte și d = 6
- PE(cuv) = valorile functiei sin pentru diferite argumente (frecvențe sau lungimi de undă)
  - E.g. Sin(2 π pos / λ_i), i =0,1,2,..,d-1

pos		$\lambda = \pi$	$\lambda = 2\pi$	$\lambda = 3\pi$	$\lambda = 4\pi$	$\lambda = 5\pi$	$\lambda = 6\pi$
0	→	sin(2*0)	sin(0)	sin(2/3*0)	sin(2/4*0)	sin(2/5*0)	sin(2/6*0)
1	→	sin(2*1)	sin(1)	sin(2/3*1)	sin(2/4*1)	sin(2/5*1)	sin(2/6*1)
2	→	sin(2*2)	sin(2)	sin(2/3*2)	sin(2/4*2)	sin(2/5*2)	sin(2/6*2)
3	→	sin(2*3)	sin(3)	sin(2/3*3)	sin(2/4*3)	sin(2/5*3)	sin(2/6*3)
4	→	sin(2*4)	sin(4)	sin(2/3*4)	sin(2/4*4)	sin(2/5*4)	sin(2/6*4)

Sin-based encoding

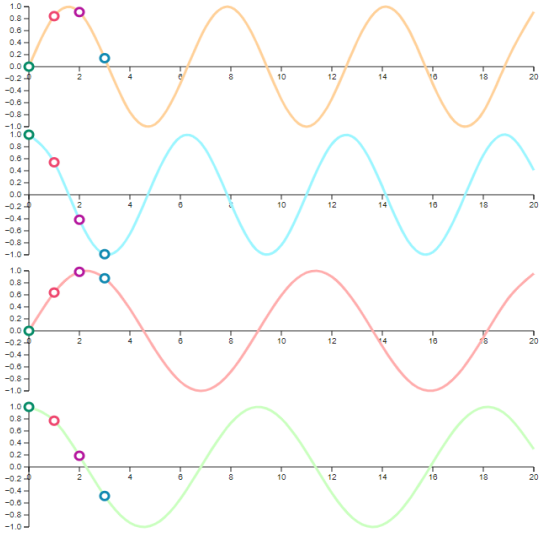


- Exemplu
  - O propoziție cu 5 cuvinte și d = 6
  - PE(cuv) = valorile functiei sin pentru diferite argumente (frecvențe sau lungimi de undă)
    - E.g. Sin(2 π pos / λ_i), i =0,1,2,..,d-1

pos		$\lambda = \pi$	$\lambda = 2\pi$	$\lambda = 3\pi$	$\lambda = 4\pi$	$\lambda = 5\pi$	$\lambda = 6\pi$
0	→	sin(0)	sin(0)	sin(0)	sin(0)	sin(0)	sin(0)
1	→	sin(2*1)	sin(1)	sin(2/3*1)	sin(2/4*1)	sin(2/5*1)	sin(2/6*1)
2	→	sin(2*2)	sin(2)	sin(2/3*2)	sin(2/4*2)	sin(2/5*2)	sin(2/6*2)
3	→	sin(2*3)	sin(3)	sin(2/3*3)	sin(2/4*3)	sin(2/5*3)	sin(2/6*3)
4	→	sin(2*4)	sin(4)	sin(2/3*4)	sin(2/4*4)	sin(2/5*4)	sin(2/6*4)

sim(PE(word$_0$),PE(word$_k$))=0!!!

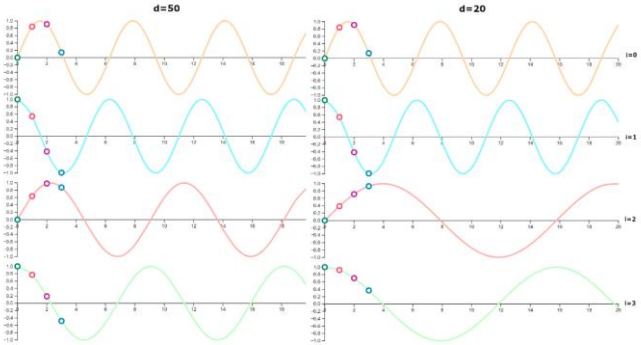
Sin&cos-based encoding



- Exemplu
  - O propoziție cu 5 cuvinte și d = 6
  - PE(cuv) = perechi (sin, cos) pentru diferite argumente (frecvențe sau lungimi de undă)
    - E.g. (sin(pos /$10000^{2i/d}$), cos(pos /$10000^{2i/d}$)), i =0,1,2,..,d-1

pos		i=0	i=1	i=2	i=3	i=4	i=5
0	→	sin(0)	cos(0)	sin(0)	cos(0)	sin(0)	cos(0)
1	→	sin(1)	cos(1)	sin(1/$10000^{4/6}$)	cos(1/$10000^{4/6}$)	sin(1/$10000^{8/6}$)	cos(1/$10000^{8/6}$)
2	→	sin(2)	cos(2)	sin(2/$10000^{4/6}$)	cos(2/$10000^{4/6}$)	sin(2/$10000^{8/6}$)	cos(2/$10000^{8/6}$)
3	→	sin(3)	cos(3)	sin(3/$10000^{4/6}$)	cos(3/$10000^{4/6}$)	sin(3/$10000^{8/6}$)	cos(3/$10000^{8/6}$)
4	→	sin(4)	cos(4)	sin(4/$10000^{4/6}$)	cos(4/$10000^{4/6}$)	sin(4/$10000^{8/6}$)	cos(4/$10000^{8/6}$)

Sin&cos-based encoding



- Resources
  - https://towardsdatascience.com/why-and-how-to-achieve-longer-context-windows-for-llms-5f76f8656ea9/
  - https://aiexpjourney.substack.com/p/an-in-depth-exploration-of-rotary-position-embedding-rope-ac351a45c794
  - https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

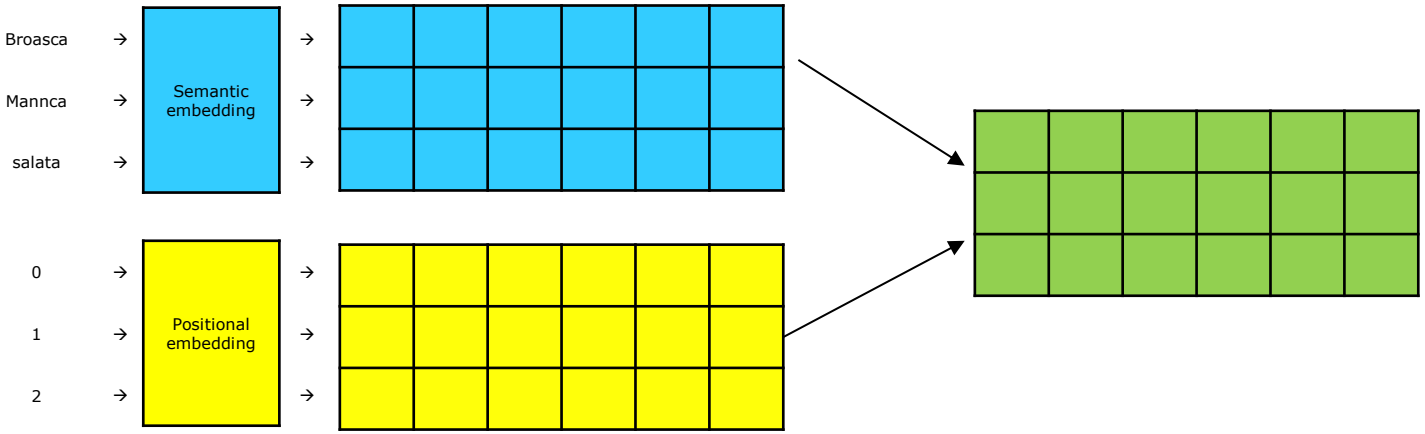
LLMs – Positional embedding

- Cuvinte → Embeddings

- Pentru o propoziție cu n cuvinte, se obțin n embedding-uri (de lungime d)
  - Care țin cont de semantica cuvintelor
    - Word Embeddings
      - descoperite / învățate
      - prin ML (Word2Vec, GloVe, etc.)
  - Care țin cont de poziția (absolută sau relativă) a cuvintelor în propoziției
    - Positional Emebeddings
      - calculate

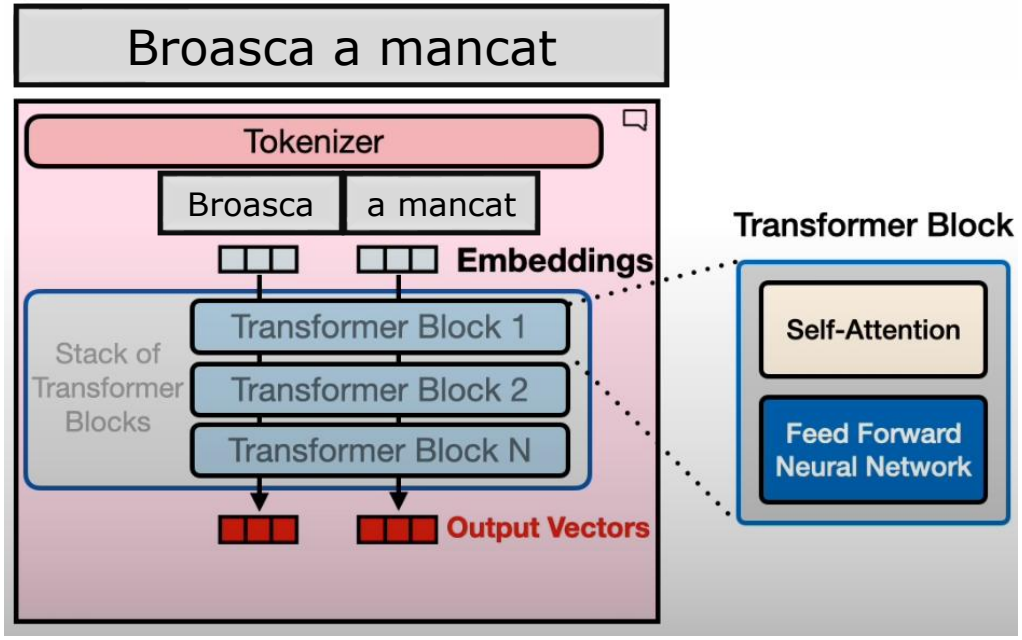
LLMs – Positional embedding

- Exemplu
  - O propoziție cu n cuvinte
  - Fiecărui cuvânt i se asociază
    - Un embedding semantic SE de lungime d
    - Un embedding pozițional PE de lungime d
    - Embedding-ul final corespunzător cuvântului va fi suma celor 2 (WE + PE) → n x d valori



Procesarea textelor/limbajelor

- Large language Models (LLMs)
  - Tokenisation
  - Embeddings
  - **Transformer & Self-attention mechanism**
  - Feed-forward network



LLMs - Mecanismul de “atentie” (attention)

flamanda

Broasca a mancat salata pentru ca ea era …

delicioasa

LLMs - Mecanismul de “atentie” (attention)

Broasca a mancat salata pentru ca ea era …

flamanda

delicioasa

Mecanismul de “atentie” (attention)

Broasca a mancat salata pentru ca ea era …

flamanda

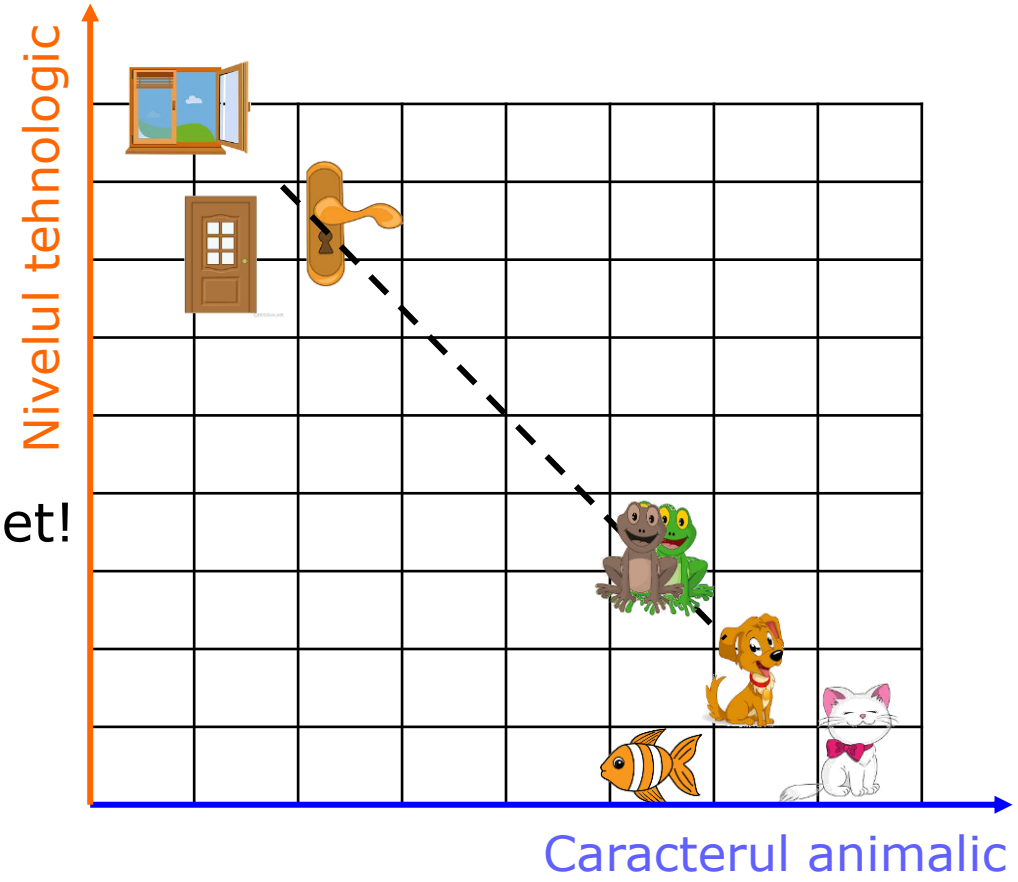
delicioasa

În acvariu era o **broască** și un pește

S-a stricat **broasca** de la ușă

Mecanismul de “atentie” (attention)

- Remember – embeddings
  - cuvinte izolate precum: *geam, ușă, clanță, pisică, pește, câine, broască*
  - Presupunem 2 atribute:
    - *caracterul animalic*
    - *nivelul tehnologic*

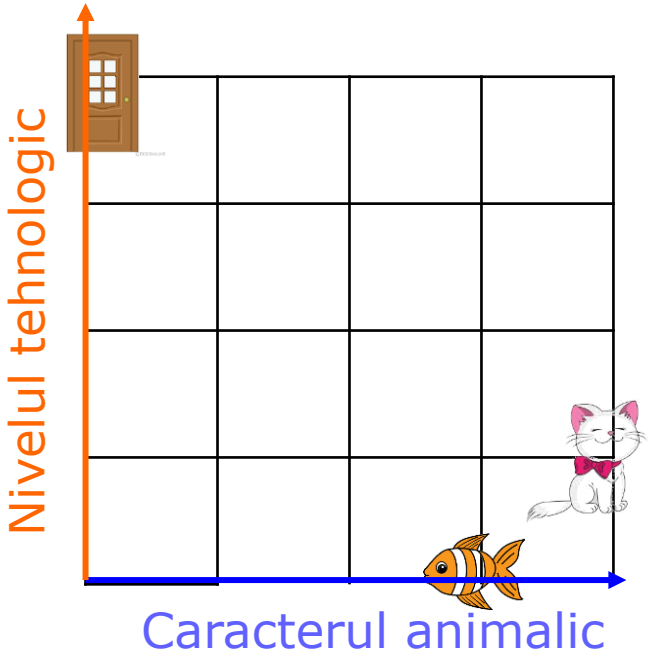


- *Attention*
  - Contextul e ca un magnet!
  - Atrage cuvintele care se potrivesc!

Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

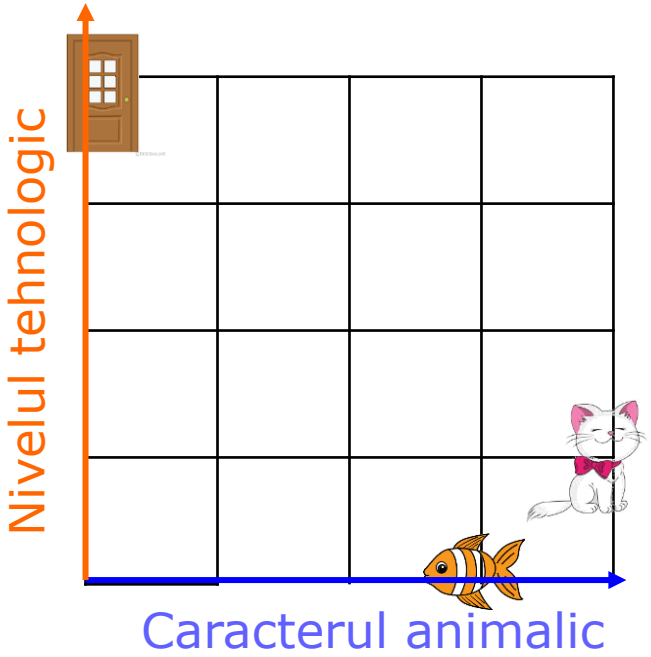


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =

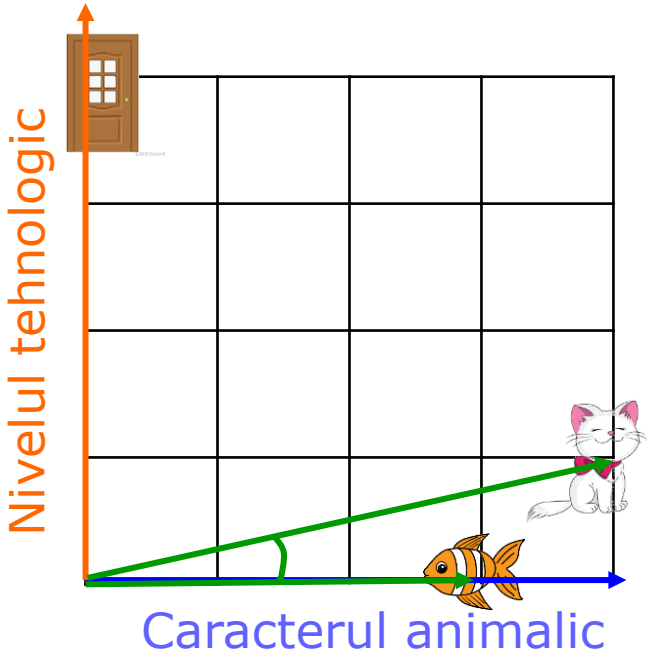


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =

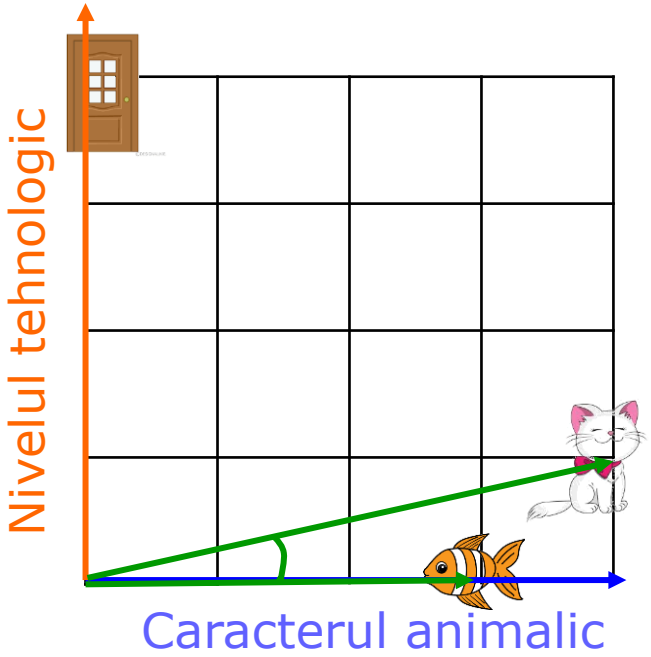


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =
  sim([4,1], [3,0]) = (4*3 + 1*0)/(√17 * √9)
  = 12 / (4.12 * 3)= 0.97

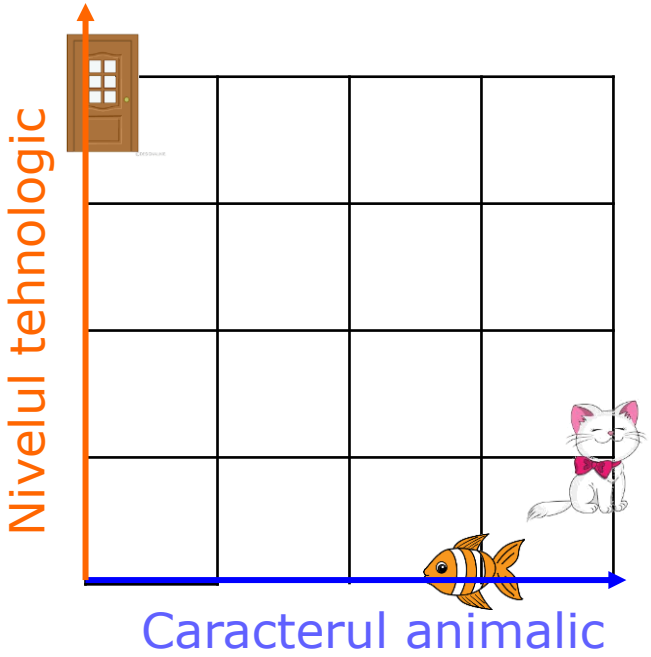


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =
  sim([4,1], [3,0]) = (4*3 + 1*0)/(√17 * √9)
  = 12 / (4.12 * 3)= 0.97
- sim(pisică, ușă) =

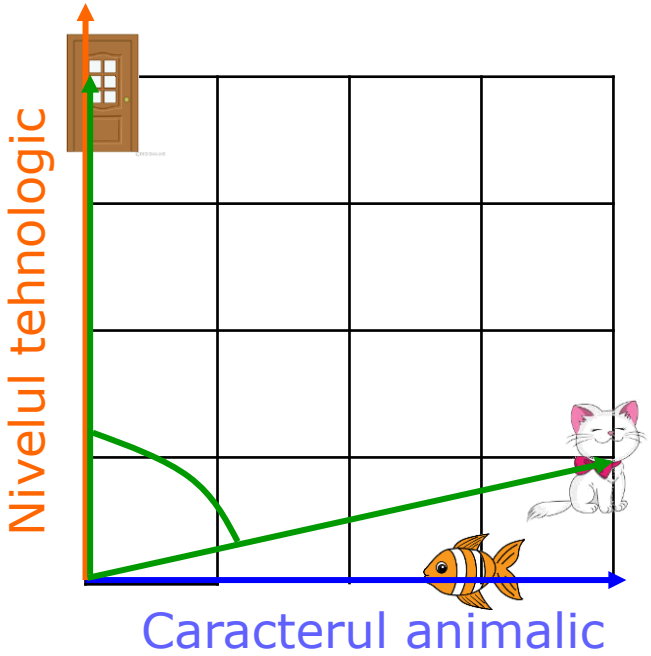


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =
  sim([4,1], [3,0]) = (4*3 + 1*0)/(√17 * √9)
  = 12 / (4.12 * 3)= 0.97
- sim(pisică, ușă) =

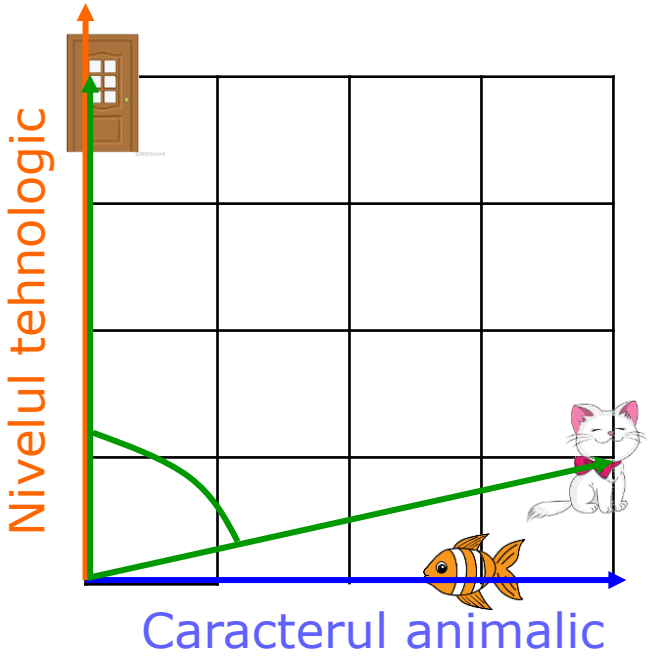


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =
  sim([4,1], [3,0]) = (4*3 + 1*0)/(√17 * √9)
  = 12 / (4.12 * 3)= 0.97
- sim(pisică, ușă) =
  sim([4,1],[0,4]) = (4*0 + 1*4) / (√17 * √16)
  = 4 / (4.12 * 4) = 0.24

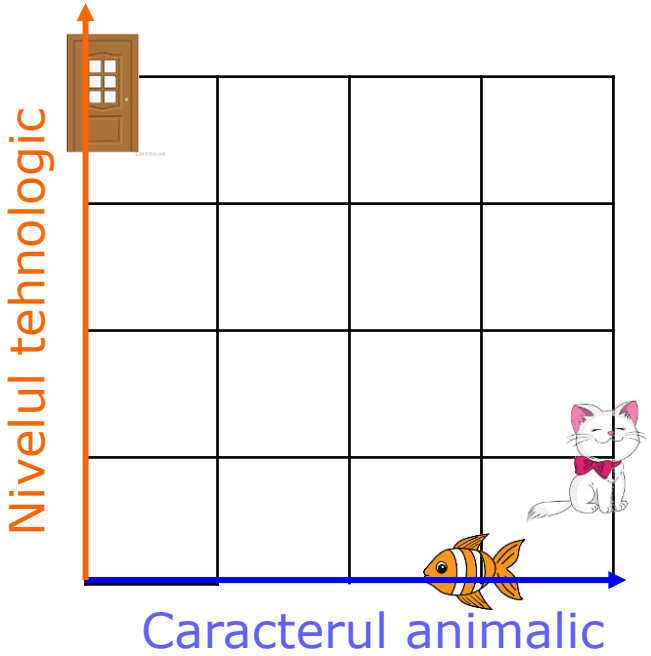


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =
  sim([4,1], [3,0]) = (4*3 + 1*0)/(√17 * √9)
  = 12 / (4.12 * 3)= 0.97
- sim(pisică, ușă) =
  sim([4,1],[0,4]) = (4*0 + 1*4) / (√17 * √16)
  = 4 / (4.12 * 4) = 0.24
- sim(pește, ușă) =

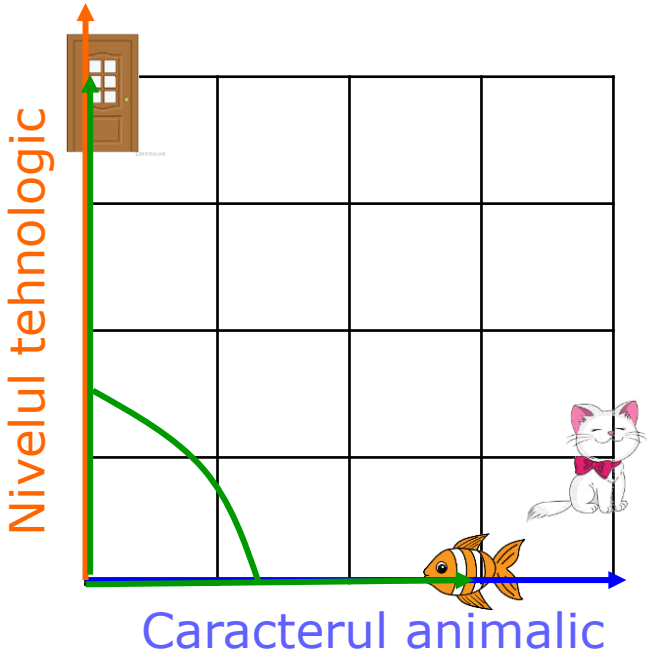


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

- sim (pisică, pește) =
  sim([4,1], [3,0]) = (4*3 + 1*0)/(√17 * √9)
  = 12 / (4.12 * 3)= 0.97
- sim(pisică, ușă) =
  sim([4,1],[0,4]) = (4*0 + 1*4) / (√17 * √16)
  = 4 / (4.12 * 4) = 0.24
- sim(pește, ușă) =

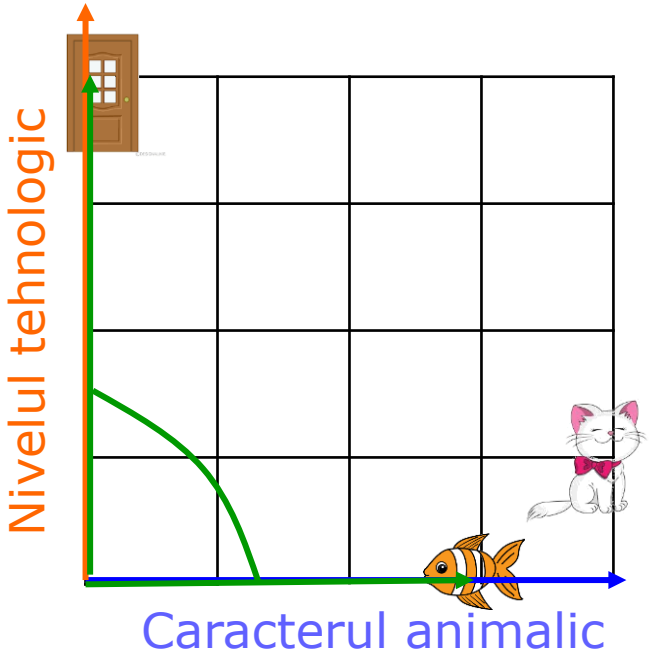


Mecanismul de “atentie” (attention)

- Similaritatea între cuvinte

Cuvântul	***caracterul animalic***	***nivelul tehnologic***
Pisică	4	1
Pește	3	0
Ușă	0	4

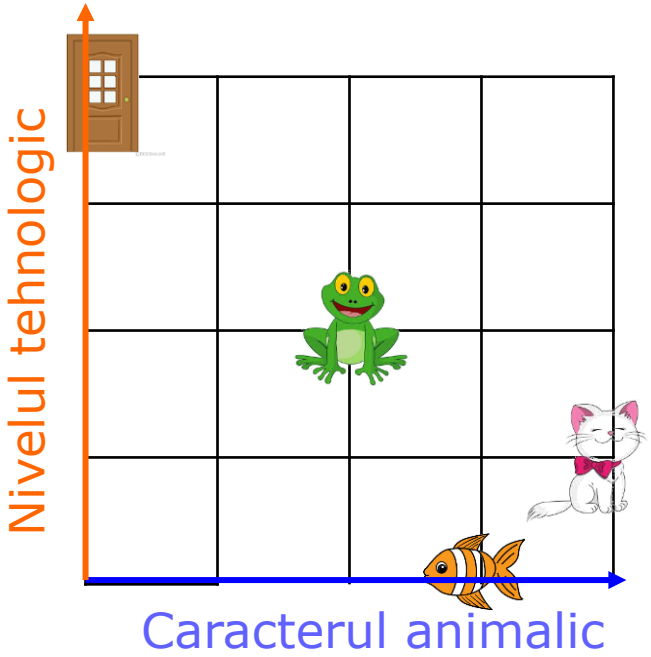
- sim (pisică, pește) =
  sim([4,1], [3,0]) = (4*3 + 1*0)/(√17 * √9)
  = 12 / (4.12 * 3)= 0.97
- sim(pisică, ușă) =
  sim([4,1],[0,4]) = (4*0 + 1*4) / (√17 * √16)
  = 4 / (4.12 * 4) = 0.24
- sim(pește, ușă) =
  sim([3,0],[0,4]) = (3*0 + 0*4) / (√9* √16)
  = 0



Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *S-a stricat **broasca** de la ușă*

	broască	ușă
broască		
ușă		

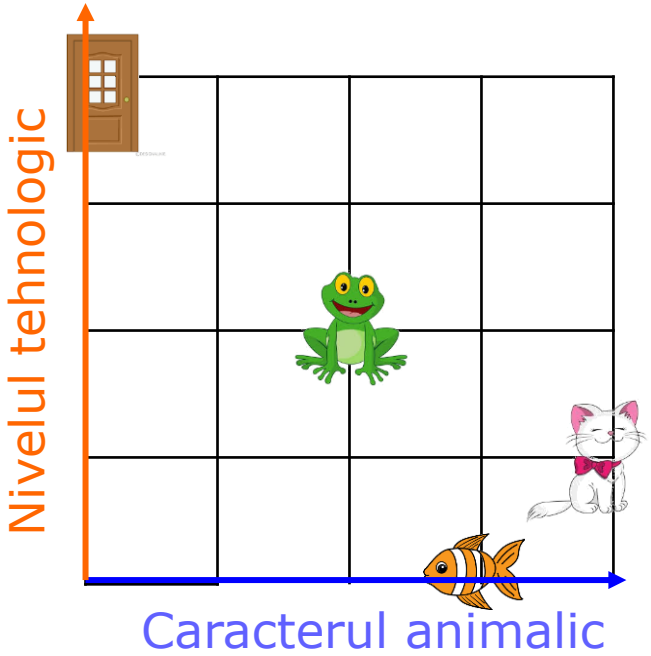


Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *S-a stricat **broasca** de la ușă*

	broască	ușă
broască		
ușă		

- sim(broască, broască) = 1
- sim(broască, ușă) = sim([2,2], [0,4]) = (2*0 + 2*4) / (√8 * √16) = 0.7

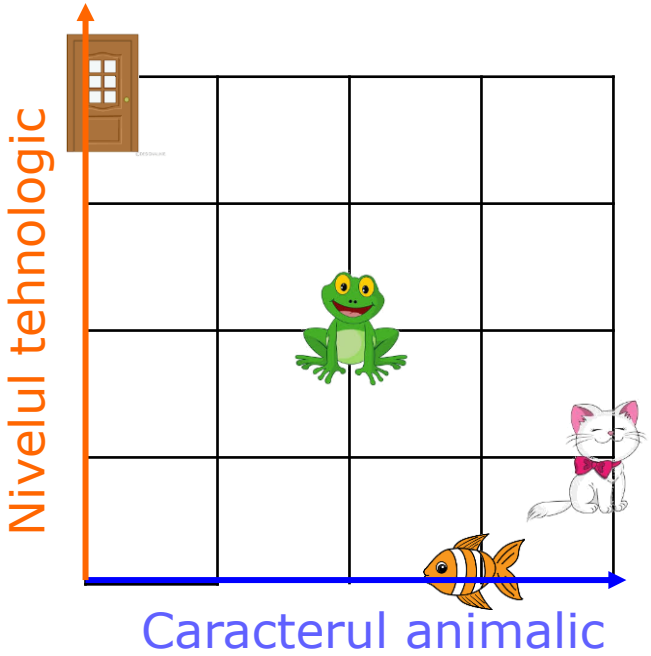


Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *S-a stricat **broasca** de la ușă*

	broască	ușă
broască	1.0	0.7
ușă	0.7	1.0

- sim(broască, broască) = 1
- sim(broască, ușă) = sim([2,2], [0,4]) = (2*0 + 2*4) / (√8 * √16) = 0.7



Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *S-a stricat **broasca** de la ușă*

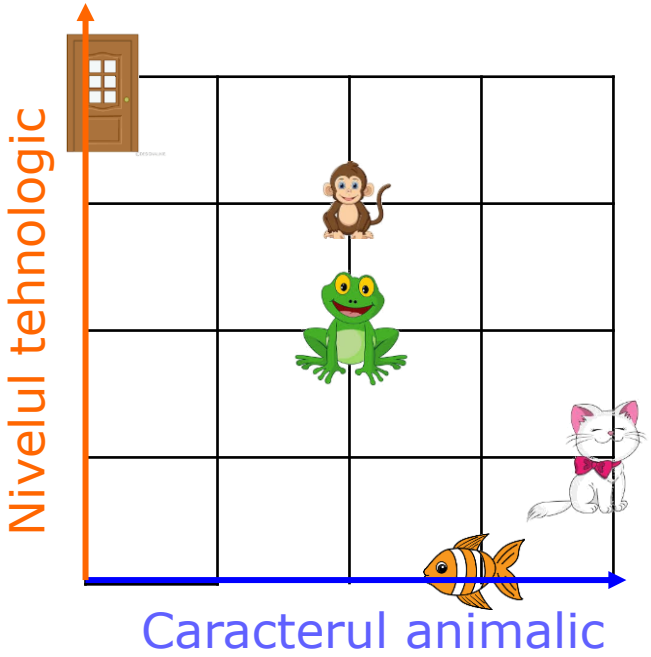
	broască	ușă
broască	1.0	0.7
ușă	0.7	1.0

- broască = 1.0 * broască + 0.7 * ușă

Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *O **broască** s-a întâlnit cu o maimuță*

	Broască	Maimuță
Broască		
Maimuță		



Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *O **broască** s-a întâlnit cu o maimuță*

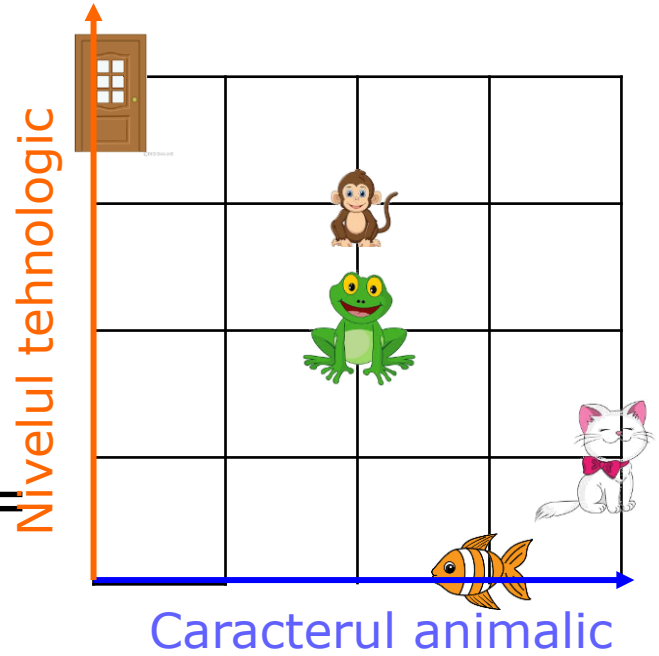
	Broască	Maimuță
Broască		
Maimuță		

- sim(broască, broască) = 1
- sim(broască, maimuță) =

  sim([2,2], [2,3]) =

  (2*2 + 2*3) / (√8 * √13) =

  12 /√2 = 0.9



Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *O **broască** s-a întâlnit cu o maimuță*

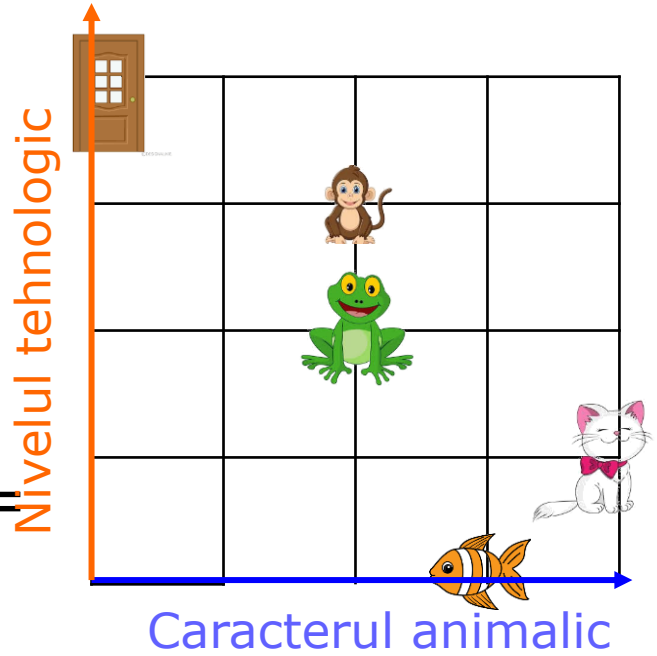
	Broască	Maimuță
Broască	1.0	0.9
Maimuță	0.9	1.0

- sim(broască, broască) = 1
- sim(broască, maimuță) =

  sim([2,2], [2,3]) =

  (2*2 + 2*3) / (√8 * √13) =

  12 /√2 = 0.9



Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *O **broască** s-a întâlnit cu o maimuță*

	broască	Maimuță
Broască	1.0	0.9
Maimuță	0.9	1.0

- broască = 1.0 * broască + 0.9 * maimuță

Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *O **broască** s-a întâlnit cu o maimuță*
  - *S-a stricat **broasca** de la ușă*

broască = 1 * broască + 0.9 * maimuță

broască = 1 * broască + 0.7 * ușă

Normalizare – de care?

broască = ? * broască + ? * maimuță

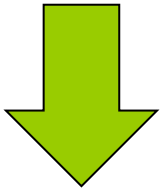
broască = ? * broască + ? * ușă

Mecanismul de “atentie” (attention)

- Contextul cuvintelor – matricea de afinitate
  - *O **broască** s-a întâlnit cu o maimuță*
  - *S-a stricat **broasca** de la ușă*

broască = 1 * broască + 0.9 * maimuță

broască = 1 * broască + 0.7 * ușă



broască = 0.52 * broască + 0.48 * maimuță

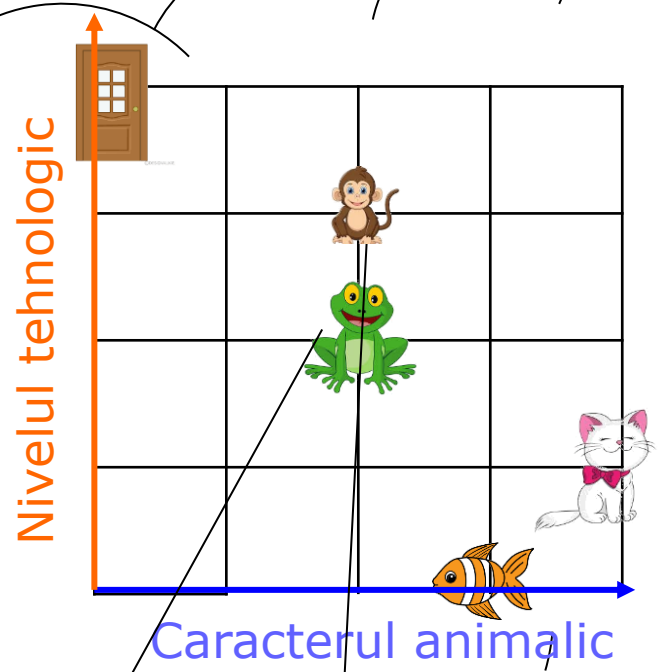
broască = 0.57 * broască + 0.43 * ușă

Mecanismul de “atentie

- Contextul cuvintelo
  - *O **broască** s-a în*
  - *S-a stricat **broasc***

broască = 1 * broas

broască = 1 * broasc



broască = 0.52 * broască + 0.48 * maimuță

broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [2,2] + 0.48 * [2,3]

broască = 0.57 * [2,2] + 0.43 * [0,4]

Mecanismul de “atentie

- Contextul cuvintelor – matri
  - *O **broască** s-a întâlnit*
  - *S-a stricat **broasca** de*

broască = 1 * broască + 0.

broască = 1 * broască + 0

broască = 0.52 * broască + 0.48

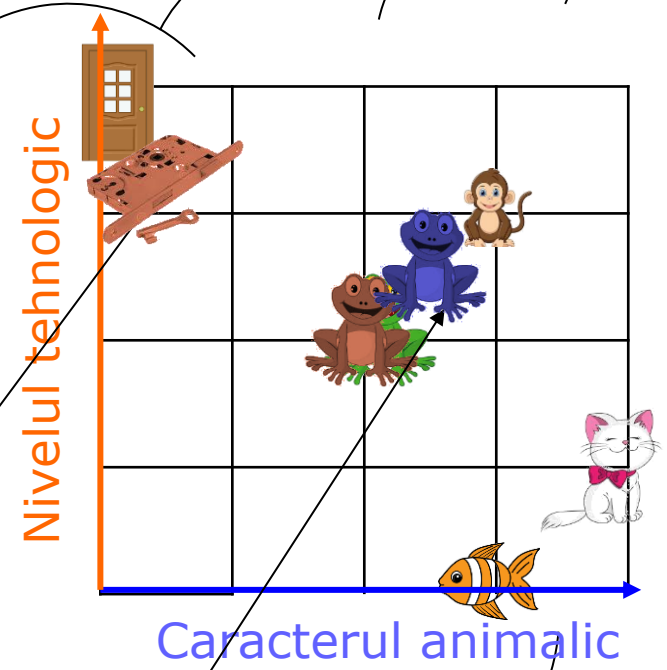
broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [2,2] + 0.48 * [2,3]

broască = 0.57 * [2,2] + 0.43 * [0,4]

broască = [2.00,2.48]

broască = [1.14,2.86]



V (Values)

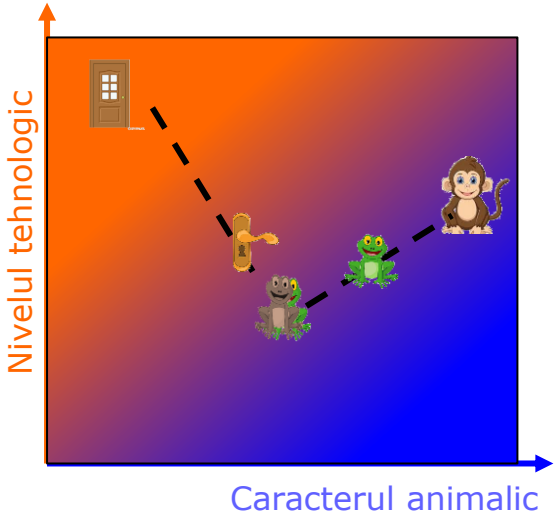
	Broască	Maimuță
Broască	0.52	0.48
Maimuță	0.48	0.52

Mecanismul de “atentie” (attention)

Un alt exemplu:

*O **broască** s-a întâlnit cu o maimuță*

*S-a stricat **broasca** de la ușă*

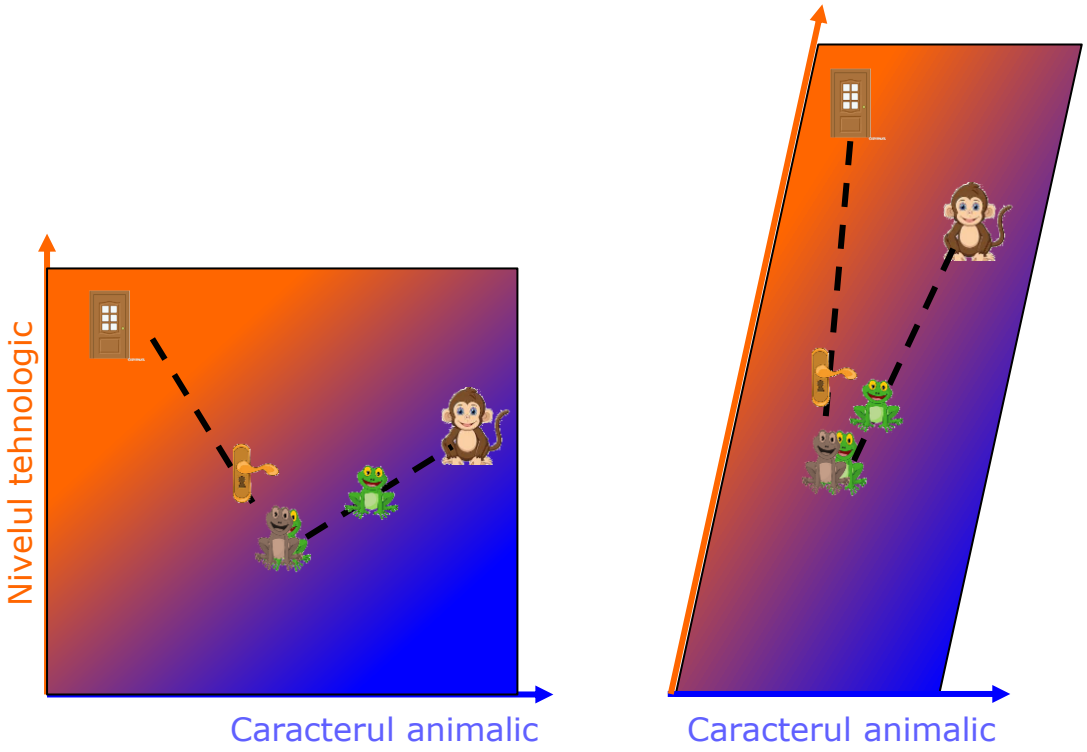


Mecanismul de “atentie” (attention)

Un alt exemplu:

*O **broască** s-a întâlnit cu o maimuță*

*S-a stricat **broasca** de la ușă*

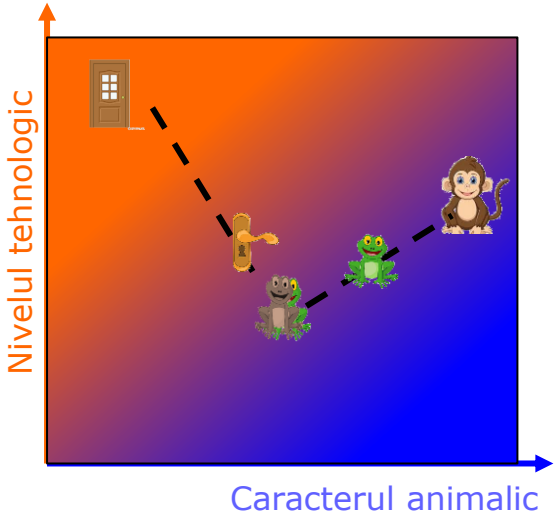


Mecanismul de “atentie” (attention)

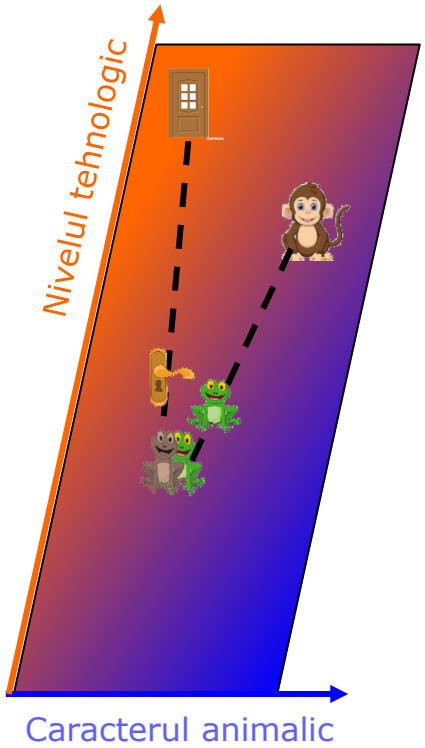
Un alt exemplu:

*O **broască** s-a întâlnit cu o maimuță*

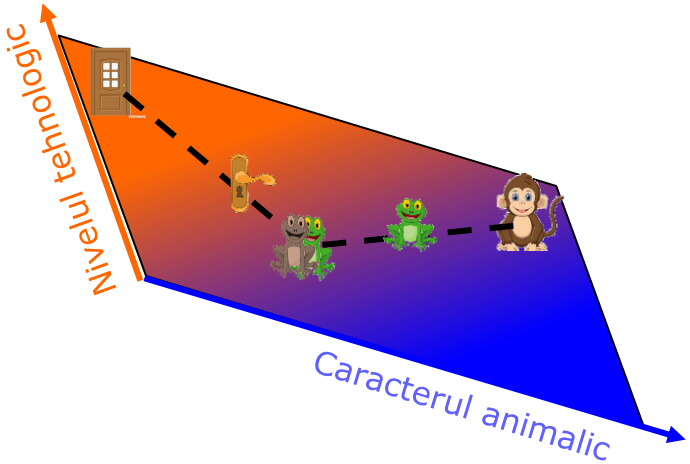
*S-a stricat **broasca** de la ușă*



a) reprez1

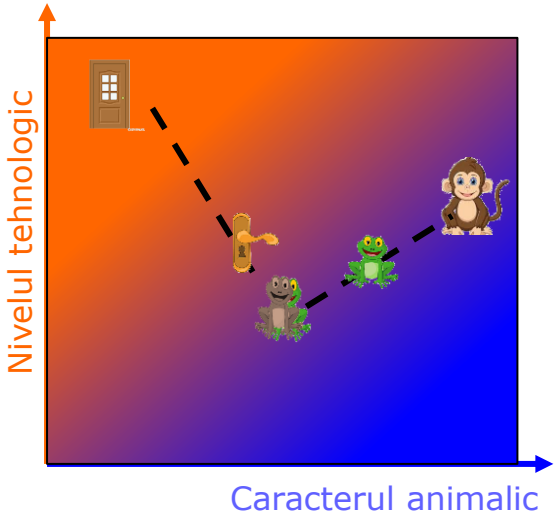


b) reprez2

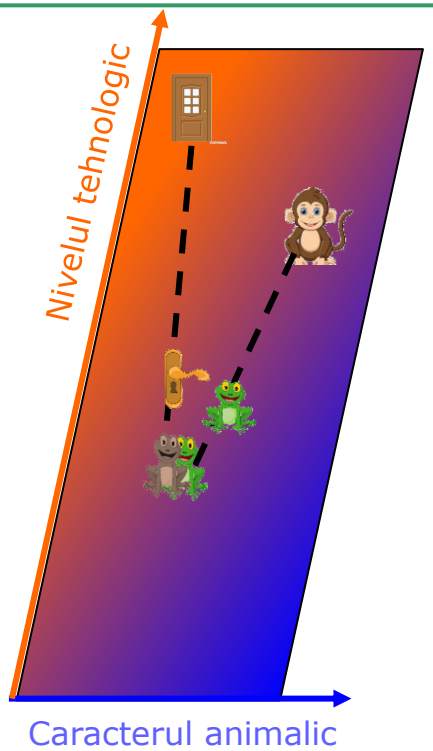


c) reprez3

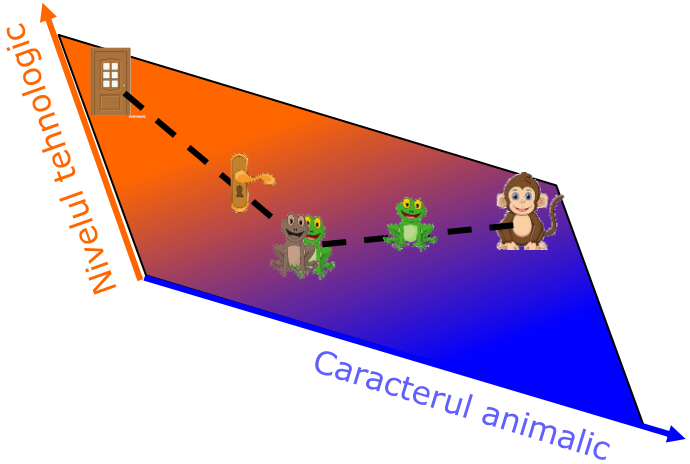
Mecanismul de “atentie” (attention)



a) reprez1



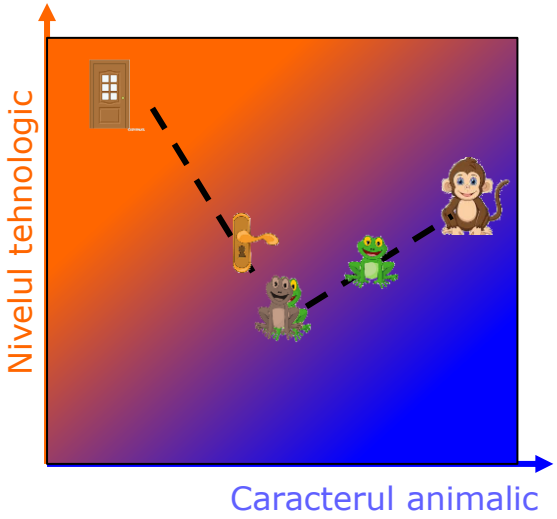
b) reprez2



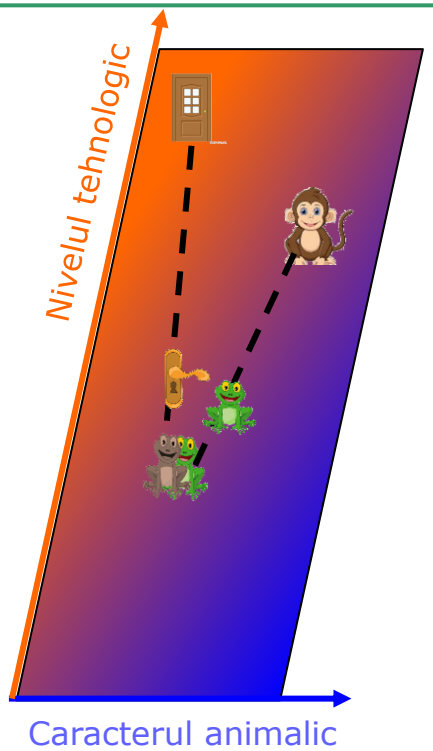
c) reprez3

Q1: reprez1 = f(reprez2) = g(reprez3) ???

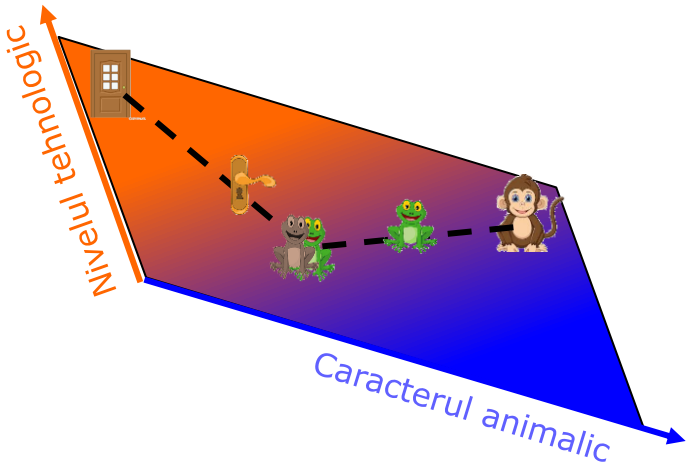
Mecanismul de “atentie” (attention)



a) reprez1



b) reprez2

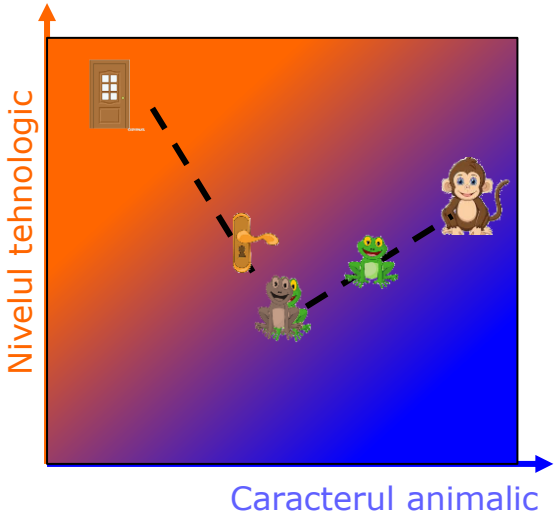


c) reprez3

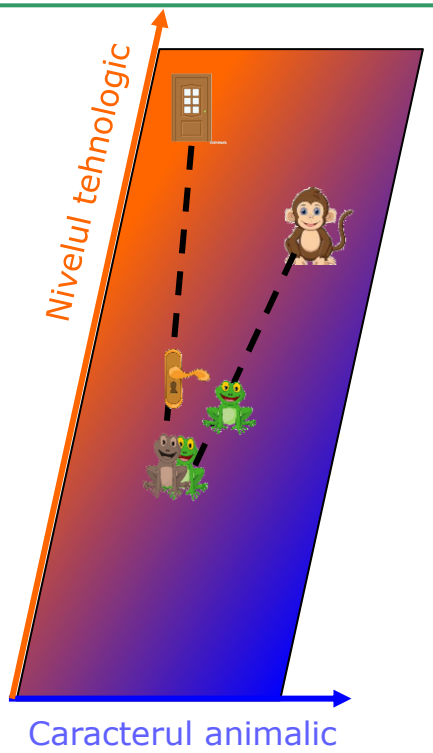
Q1: reprez1 = f(reprez2) = g(reprez3) ???

Da!!! Tranformare liniară!!!

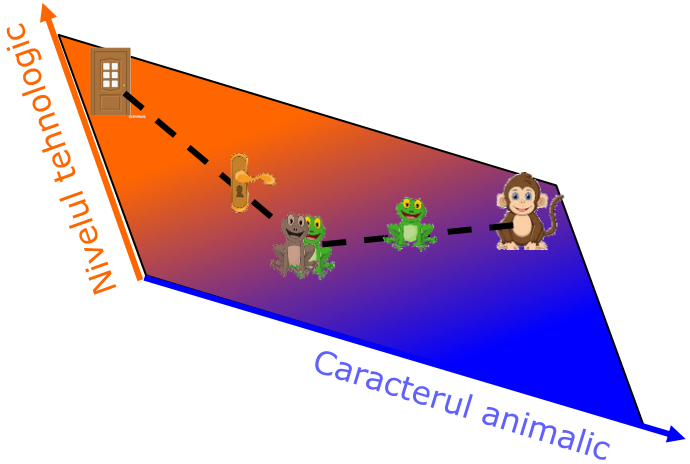
Mecanismul de “atentie” (attention)



a) reprez1



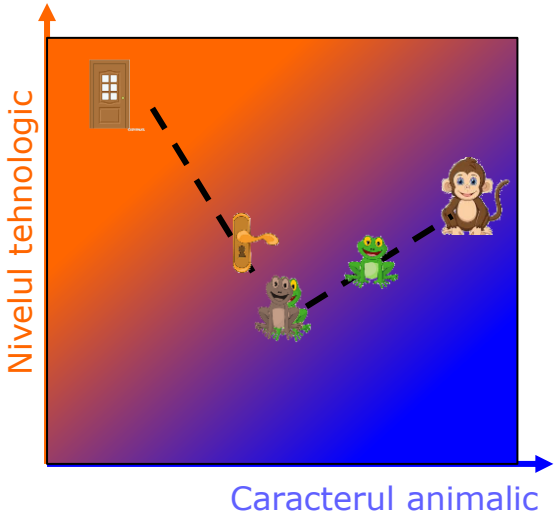
b) reprez2



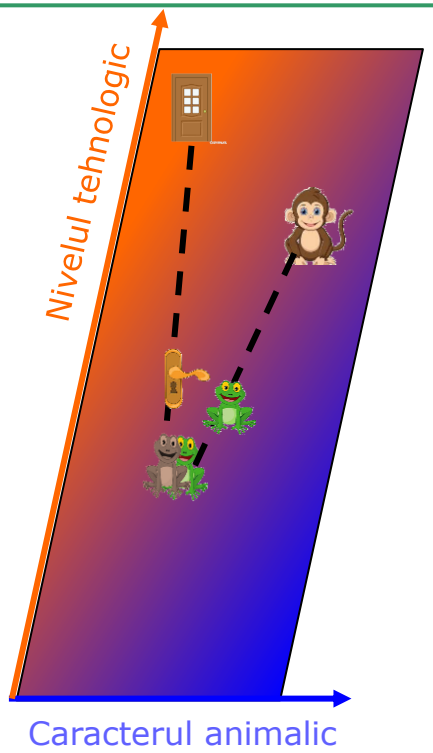
c) reprez3

Q2: Care reprezentare e mai bună pentru a stabili similaritatea (deci a face diferența între sensurile lui *broască*)?

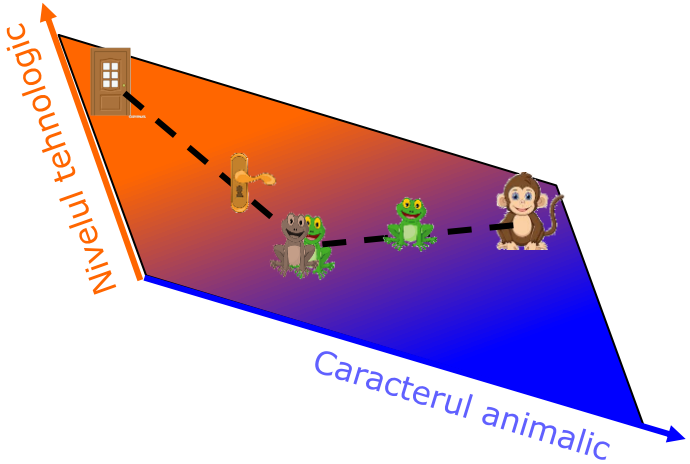
Mecanismul de “atentie” (attention)



a) reprez1



b) reprez2



c) reprez3

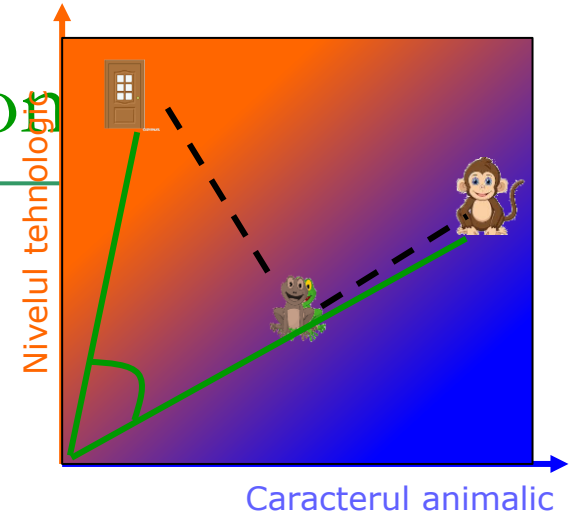
Q2: Care reprezentare e mai bună pentur a stabili similaritatea (deci a face diferența între sensurile lui *broască*)?

Reprezentarea 1 sau 3!

Mecanismul de “atentie” (attention

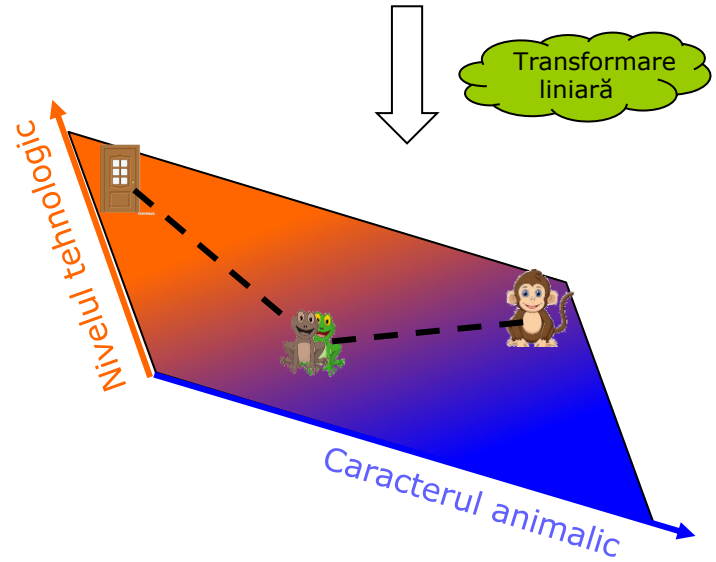
- Sim(ușă, maimuță) =

  sim(,)=



- Sim(ușă, maimuță) =

  sim(,)=

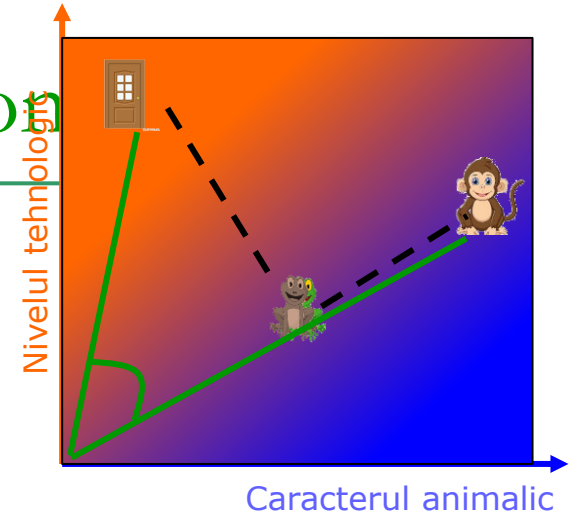


Transformare liniară

Mecanismul de “atentie” (attention

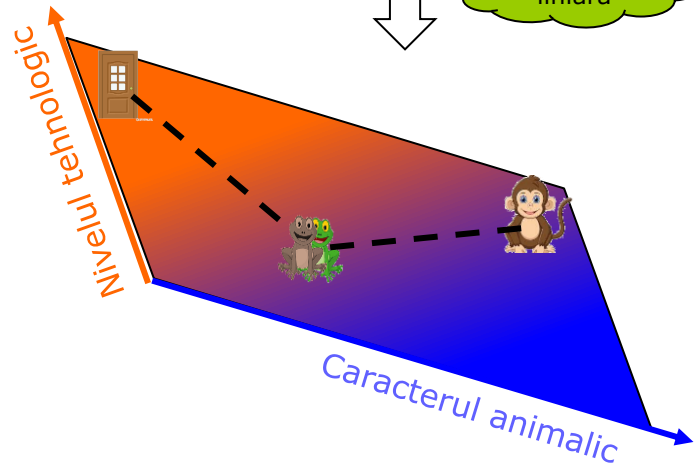
- Sim(ușă, maimuță) =

  sim(,) =



- Sim(ușă, maimuță) =

  sim(,) =



K (Key) =

Q (Queries) =

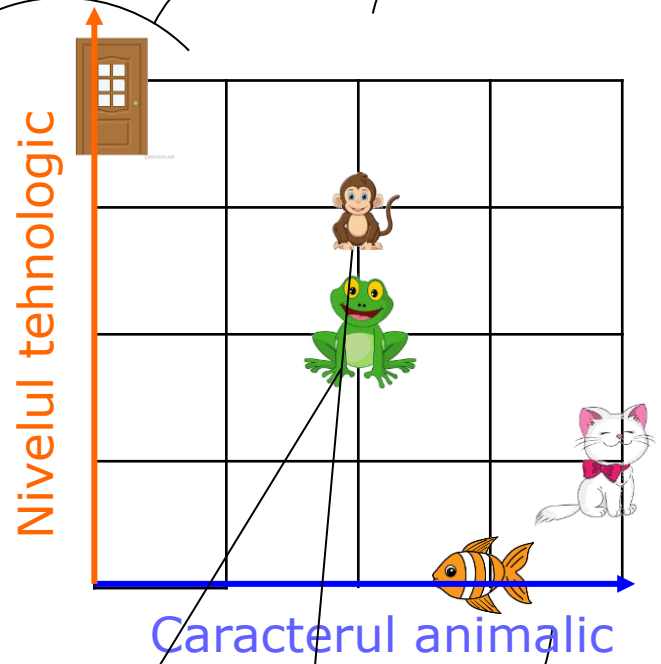
Valorile optime - ANN

Mecanismul de “atentie

- Contextul cuvintelo
  - *O **broască** s-a în*
  - *S-a stricat **broasc***

broască = 1 * broas

broască = 1 * broasc



broască = 0.52 * broască + 0.48 * maimuță

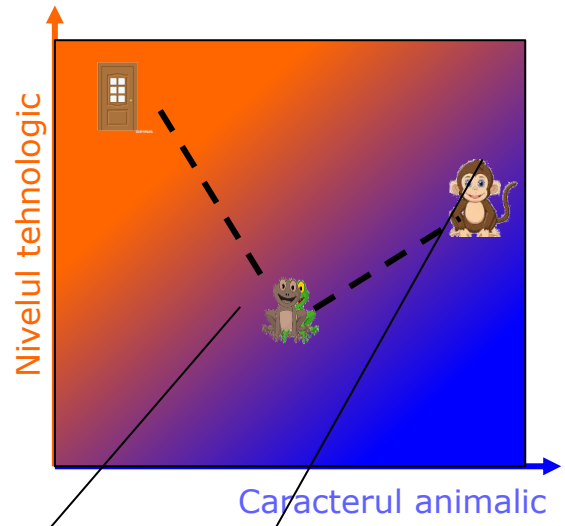
broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [2,2] + 0.48 * [2,3]

broască = 0.57 * [2,2] + 0.43 * [0,4]

Mecanismul de “atenţie”

- Contextul cuvintelor
  - *O **broască** s-a în*
  - *S-a stricat **broasc***



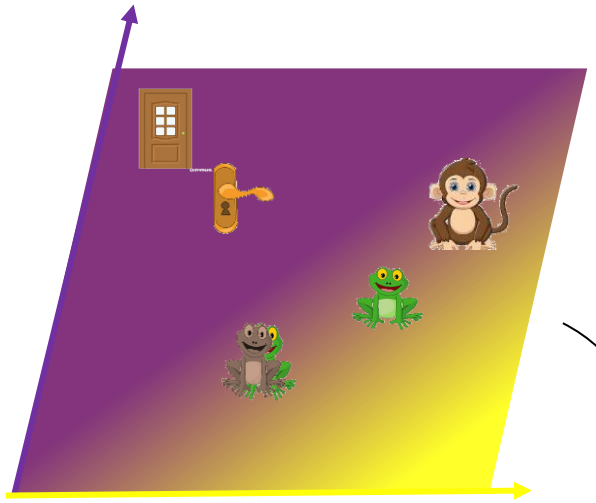
broască = 1 * broască + 0.9 * mai
broască = 1 * broască + 0.7 * ușă

broască = 0.52 * broască + 0.48 * maimuță
broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [2,2] + 0.48 * [2,3]
broască = 0.57 * [2,2] + 0.43 * [0,4]

Mecanismul de “atentie

- Contextul cuvintelo
  - *O **broască** s-a în*
  - *S-a stricat **broasc***



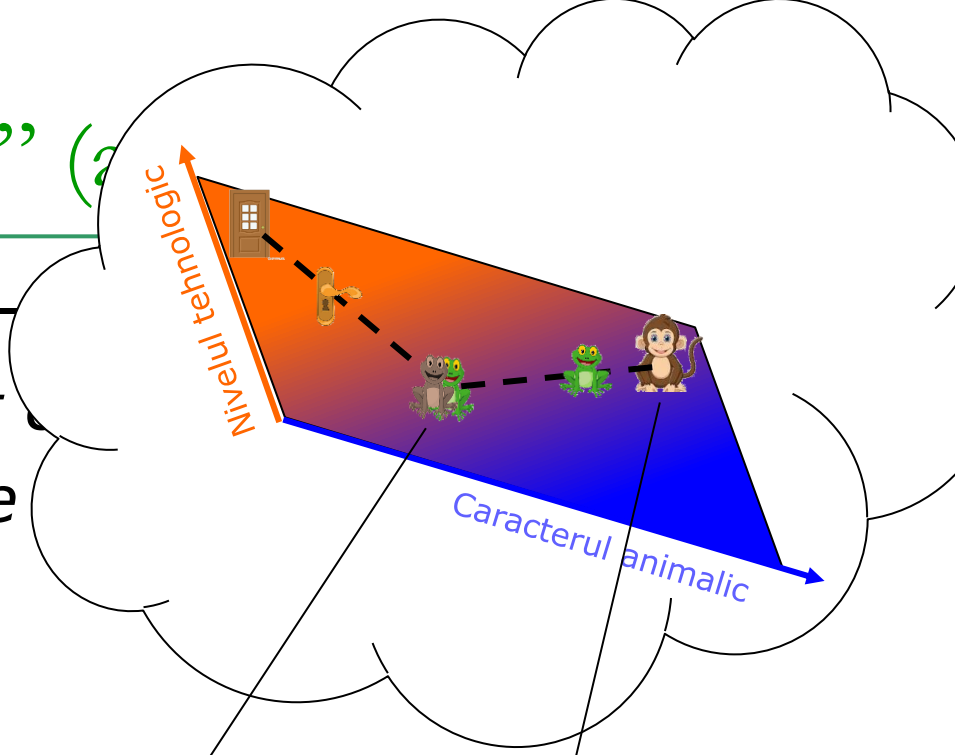
broască = 0.52 * broască + 0.48 * maimuță
broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [2,2] + 0.48 * [2,3]
broască = 0.57 * [2,2] + 0.43 * [0,4]

broască = [2.00,2.48]
broască = [1.14,2.86]

Mecanismul de “atentie” (a

- Contextul cuvintelor –
  - *O **broască** s-a întâlnit*
  - *S-a stricat **broasca** de*

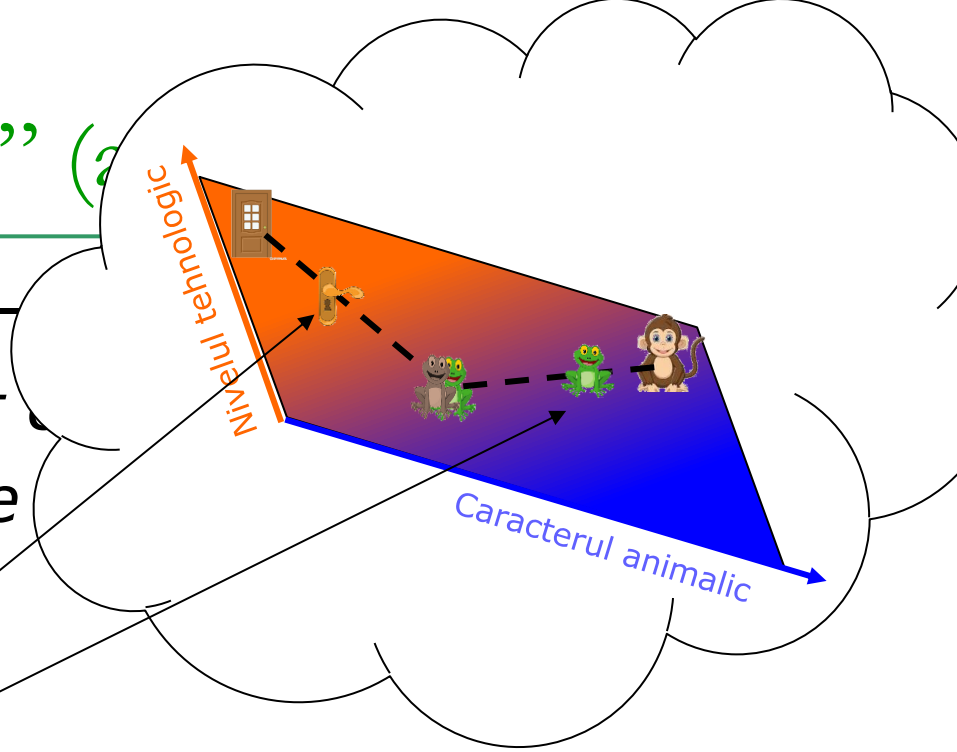


broască = 0.52 * broască + 0.48 * maimuță
broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [broască(KQ,KQ)] + 0.48 * [maimuță(KQ,KQ)]
broască = 0.57 * [broască(KQ,KQ)] + 0.43 * [ușă(KQ,KQ)]

Mecanismul de “atentie” (a

- Contextul cuvintelor –
  - *O **broască** s-a întâlnit*
  - *S-a stricat **broasca** de*

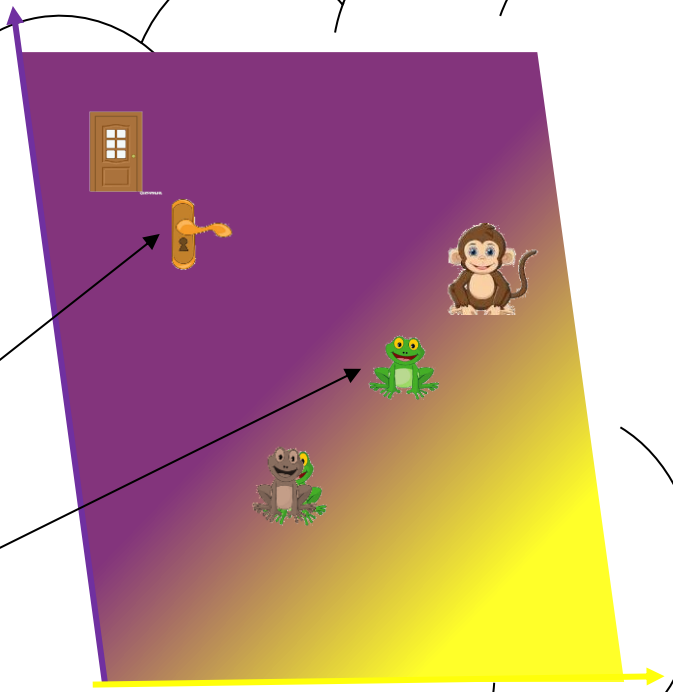


broască = 0.52 * broască + 0.48 * maimuță
broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [broască(KQ,KQ)] + 0.48 * [maimuță(KQ,KQ)]
broască = 0.57 * [broască(KQ,KQ)] + 0.43 * [ușă(KQ,KQ)]

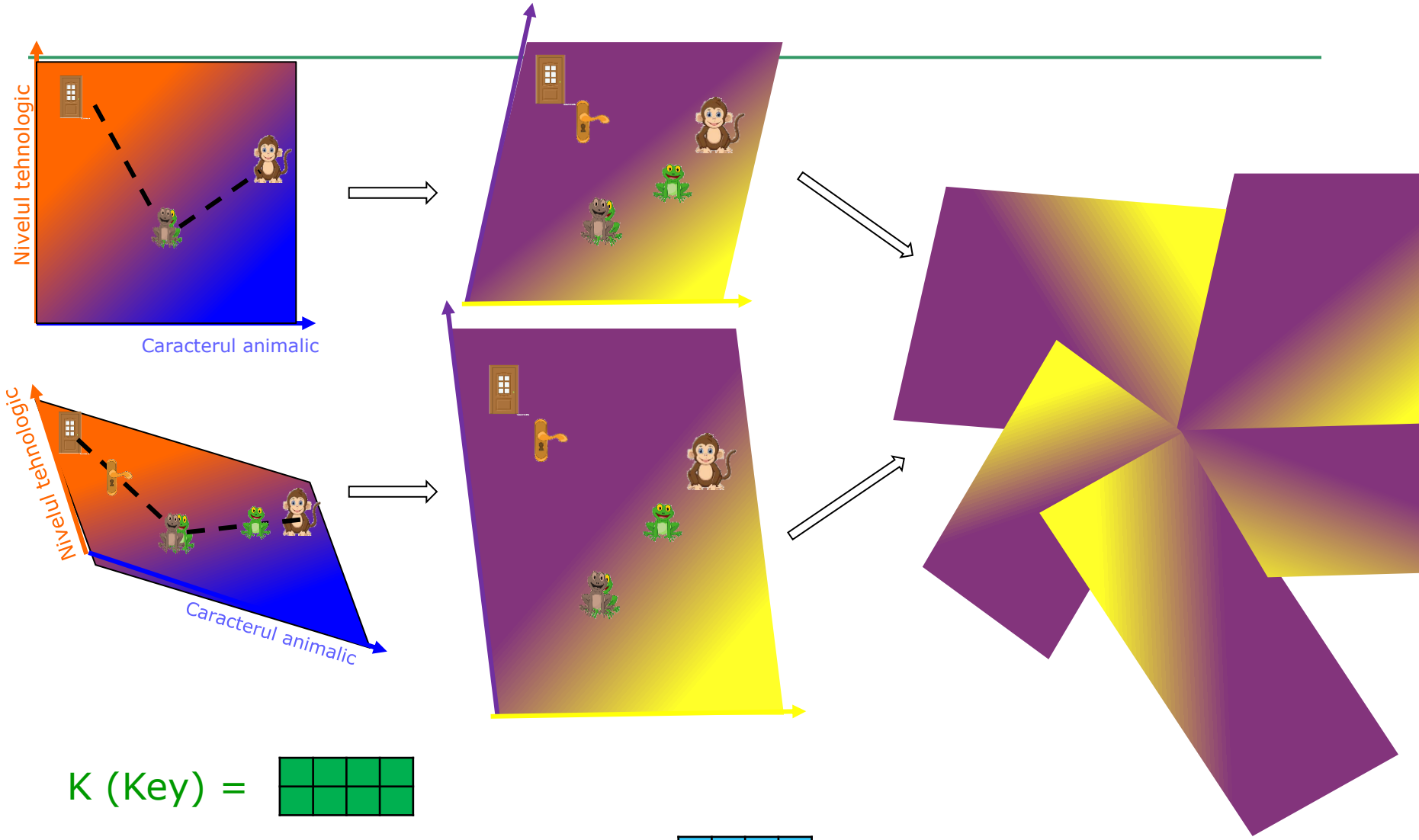
Mecanismul de “atentie” (a

- Contextul cuvintelor – m
  - *O **broască** s-a întâlnit*
  - *S-a stricat **broasca** de*



broască = 0.52 * broască + 0.48 * maimu
broască = 0.57 * broască + 0.43 * ușă

broască = 0.52 * [broască(KQ,KQ)] + 0.48 * [maimuță(KQ,KQ)]
broască = 0.57 * [broască(KQ,KQ)] + 0.43 * [ușă(KQ,KQ)]



K (Key) =

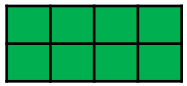
Q (Queries) =

V (Values)

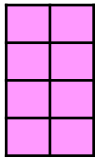
Mecanismul de “atentie” (attention)

- Identificarea celor mai bune embedding-uri pentru a stabili similaritatea intre elemente
  - Se bazează pe feature-urile elementelor (e.g. Caracterul animalic, nivelul tehnologic)

K (Key) =

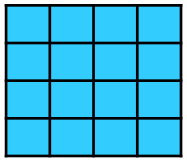


Q (Queries) =



- Identificarea celor mai bune embedding-uri pentru a prezice următorul cuvânt
  - Se bazează pe co-apariția elementelor în același context

V (Values) =



Mecanismul de “atentie” (attention)

Input	**token1**	**token2**	**token3**	**token4**	**token5**	**token6**	**token7**	**token8**	**token9**
representation	x_1								
Queries	q_1								
Keys	k_1								
Values	v_1								
Similarities	$q_1 \bullet k_1$	$q_1 \bullet k_2$	$q_1 \bullet k_3$	$q_1 \bullet k_4$	$q_1 \bullet k_5$	$q_1 \bullet k_6$	$q_1 \bullet k_7$	$q_1 \bullet k_8$	$q_1 \bullet k_9$
Scores s= [s1, s2, .., s9]	$q_1 \bullet k_1 / \sqrt{D}$	$q_1 \bullet k_2 / \sqrt{D}$	$q_1 \bullet k_3 / \sqrt{D}$	$q_1 \bullet k_4 / \sqrt{D}$	$q_1 \bullet k_5 / \sqrt{D}$	$q_1 \bullet k_6 / \sqrt{D}$	$q_1 \bullet k_7 / \sqrt{D}$	$q_1 \bullet k_8 / \sqrt{D}$	$q_1 \bullet k_9 / \sqrt{D}$
w = Softmax(s)									
Weightening the values	$z_1 = w_1 * v_1 + w_2 * v_2 + \ldots + w_9 * v_9$								

Mecanismul de “atentie” (attention)

- Interogari intr-o BD
  - Intput:
    - O multime de recorduri { (key_i, $value_i$), i =1,2, …, m}
    - Un query q
  - Output
    - $value_j$ a.i. q e similar cu key_j
  - Exemplu
    - DB = {(Apostol, Mihai), (Baciu, Alina), (Cretu, Gabriel), (Pop, Ionica),(Popa, Madalina), (Popescu, Cosmina)}
    - Query = “Pop”
    - Answer:

Mecanismul de “atentie” (attention)

- Interogari intr-o BD
  - Intput:
    - O multime de recorduri { (key_i, $value_i$), i =1,2, …, m}
    - Un query q
  - Output
    - $value_j$ a.i. q e similar cu key_j
  - Exemplu
    - DB = {(Apostol, Mihai), (Baciu, Alina), (Cretu, Gabriel), (Pop, Ionica),(Popa, Madalina), (Popescu, Cosmina)}
    - Query = “Pop”
    - Answer:
      - Ionica – perfect match (e.g. Hamming Distance)
      - Ionica, Madalina, Cosmina – partial match (e.g. Levenshtein Distance), top-k raspunsuri

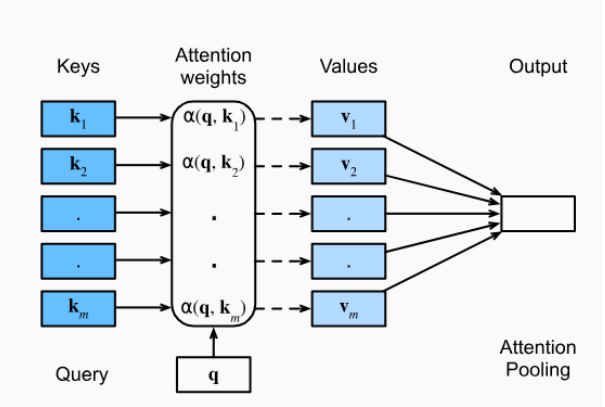
Q=Pop	Apostol	Baciu	Cretu	Pop	Popa	Popescu
Hamming	6	5	5	0	4	7
Levenshtein	6	5	5	0	1	4

Mecanismul de “atentie” (attention)

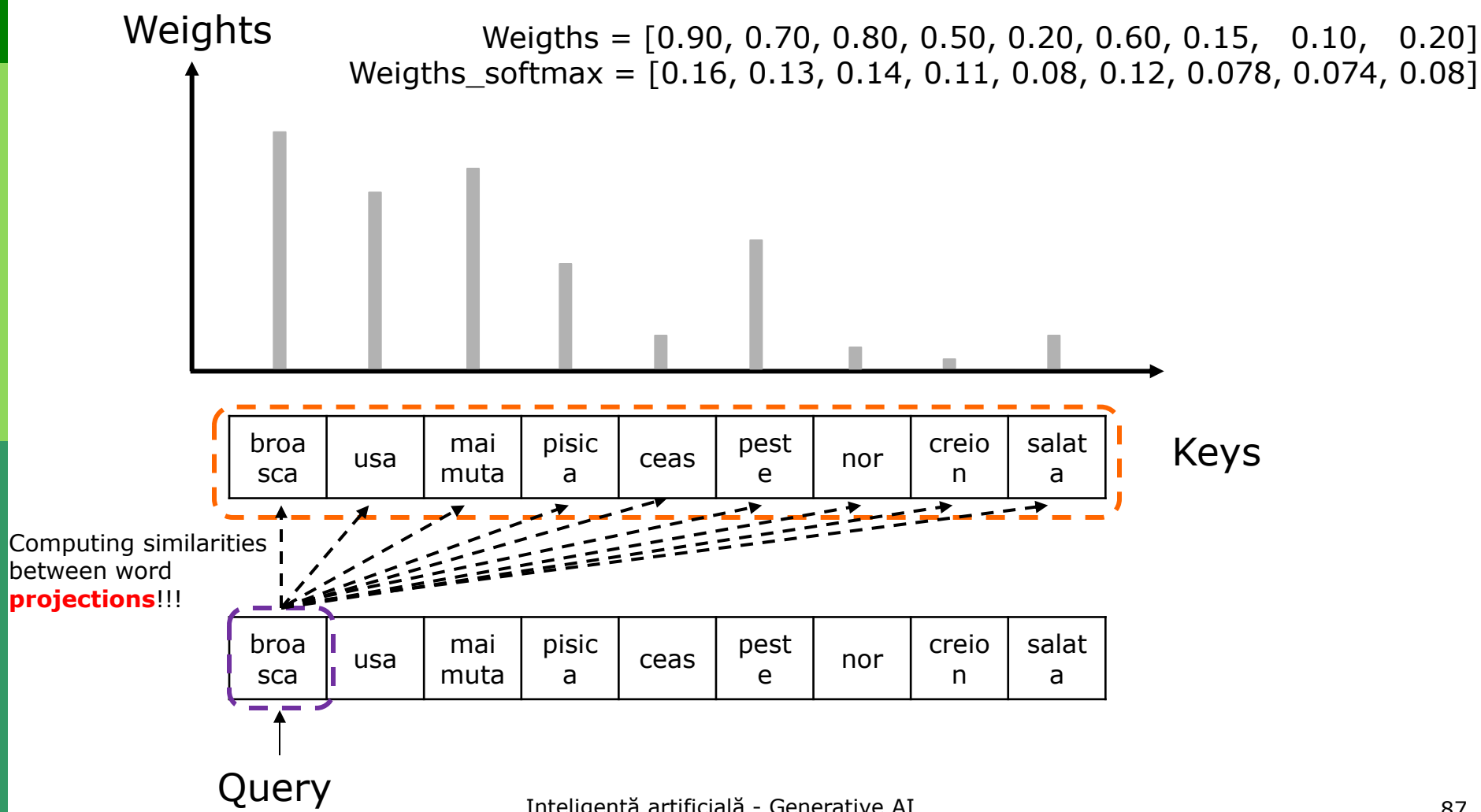
- Interogari intr-o baza de date
  - Exemplu - text
    - DB = {(Apostol, Mihai), (Baciu, Alina), (Cretu, Gabriel), (Pop, Ionica),(Popa, Madalina), (Popescu, Cosmina)}
    - Q = “Pop”
    - Answer:
      - {Ionica} – perfect match (e.g. Hamming Distance)
      - {Ionica, Madalina, Cosmina} – partial match (e.g. Levenshtein Distance), top-k raspunsuri

Q=Pop	Apostol	Baciu	Cretu	Pop	Popa	Popescu
Hamming	6	5	5	0	4	7
Levenshtein	6	5	5	0	1	4

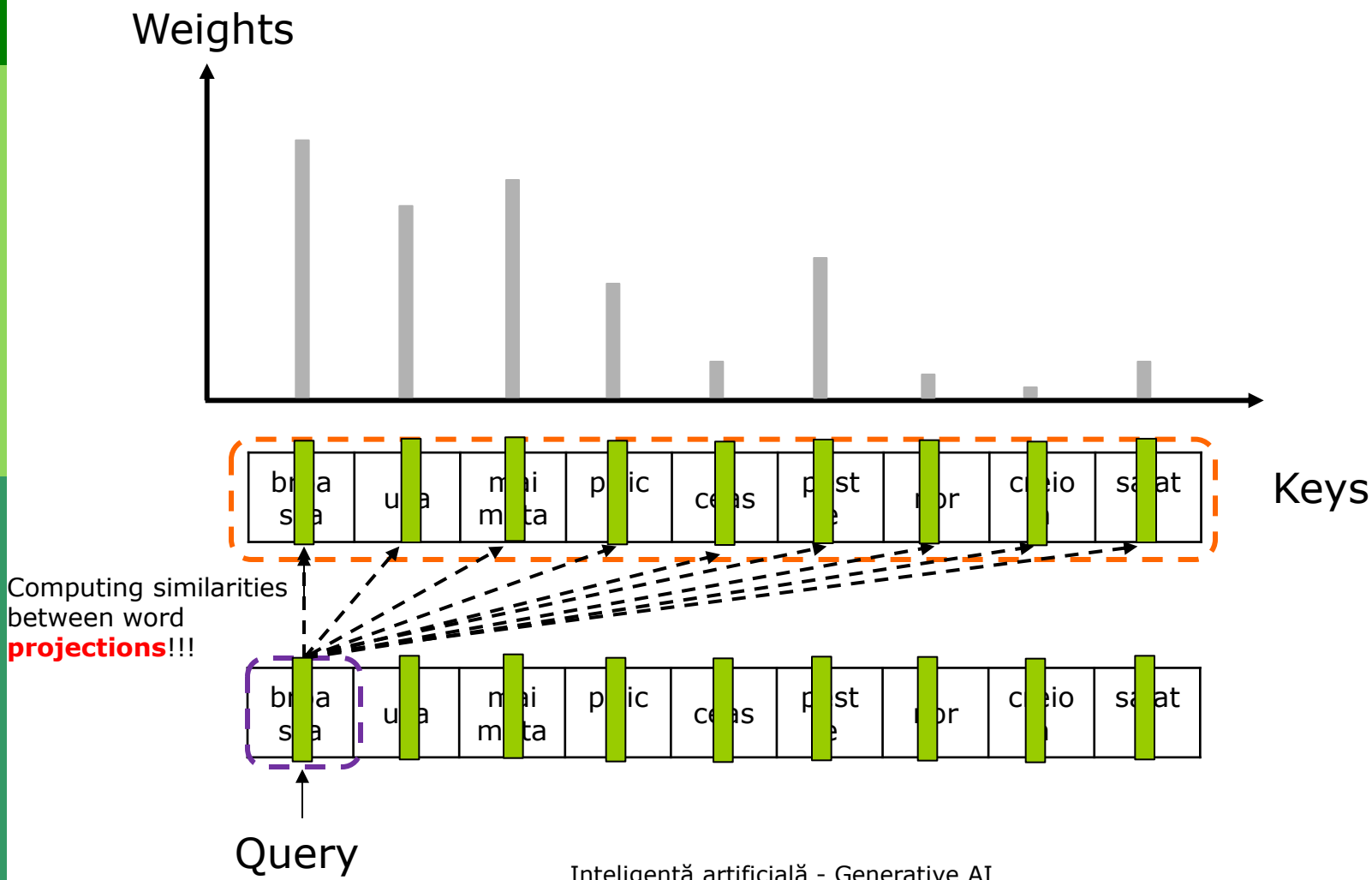
  - Mecanismul de atentie
    - Attention(q,DB) = $\Sigma\ \alpha(q,key_i)*value_i$,
      - Query – focusul current (query-ul pe care modelul il adreseaza in legatura cu un cuvant/token)
      - Key – label-ul sau referinta unui cuvant
      - Value - Informatia actuala codata de un cuvant
    - $\alpha(q,key_i)$ – attention weights
      - Daca un singur coefficient e 1 si restul 0 → interogari
      - Daca toti coeficientii α = 1/m → Average Pooling
      - Pentru a avea coeficienti α ca ponderi (weights)
        - $\alpha'(q,key_i) = \alpha(q,key_i) / \Sigma\ \alpha(q,key_j)$
        - $\alpha'(q,key_i) = \exp(\alpha(q,key_i)) / \Sigma \exp(\alpha(q,key_j))$



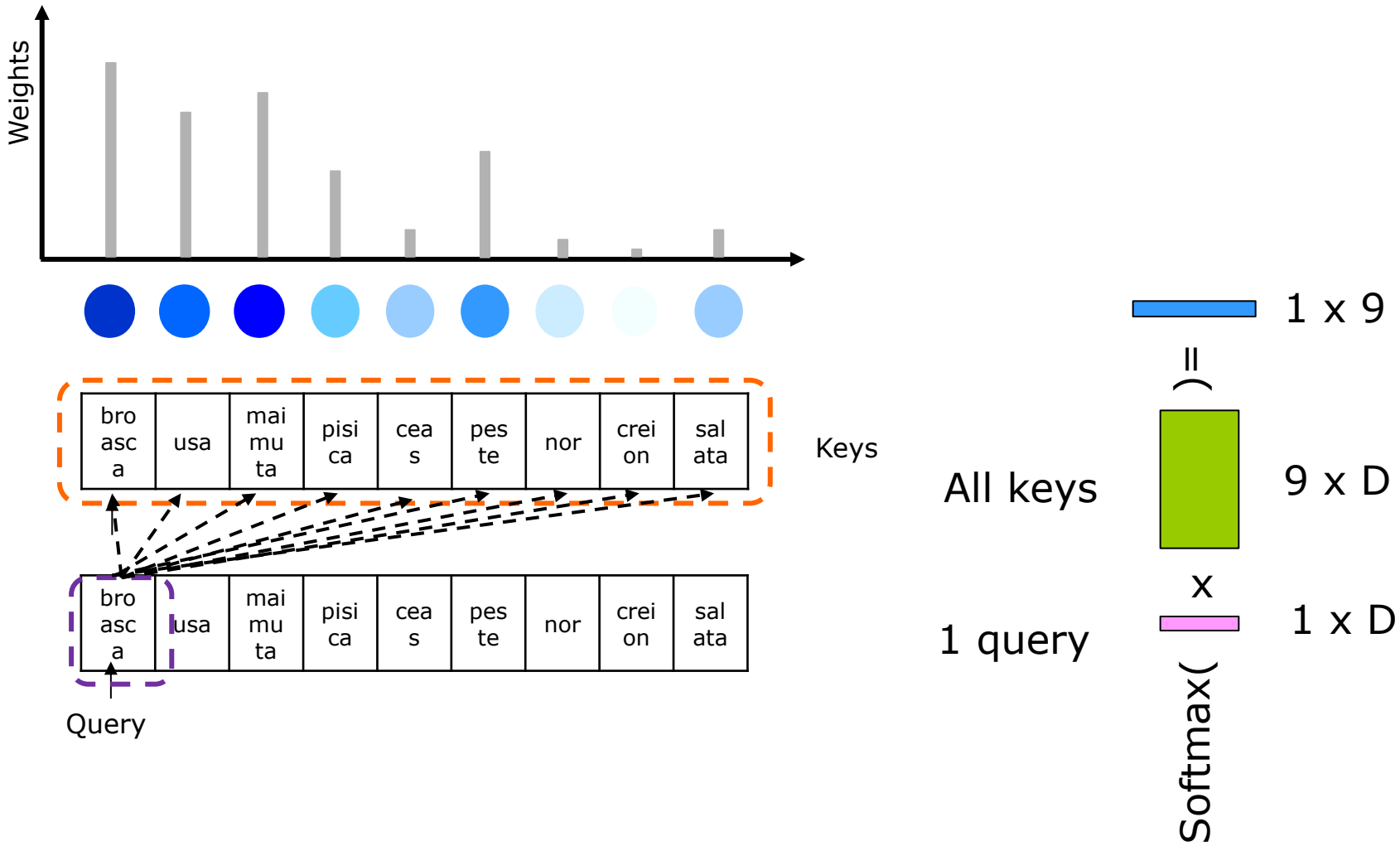
Mecanismul de “atentie” (attention)



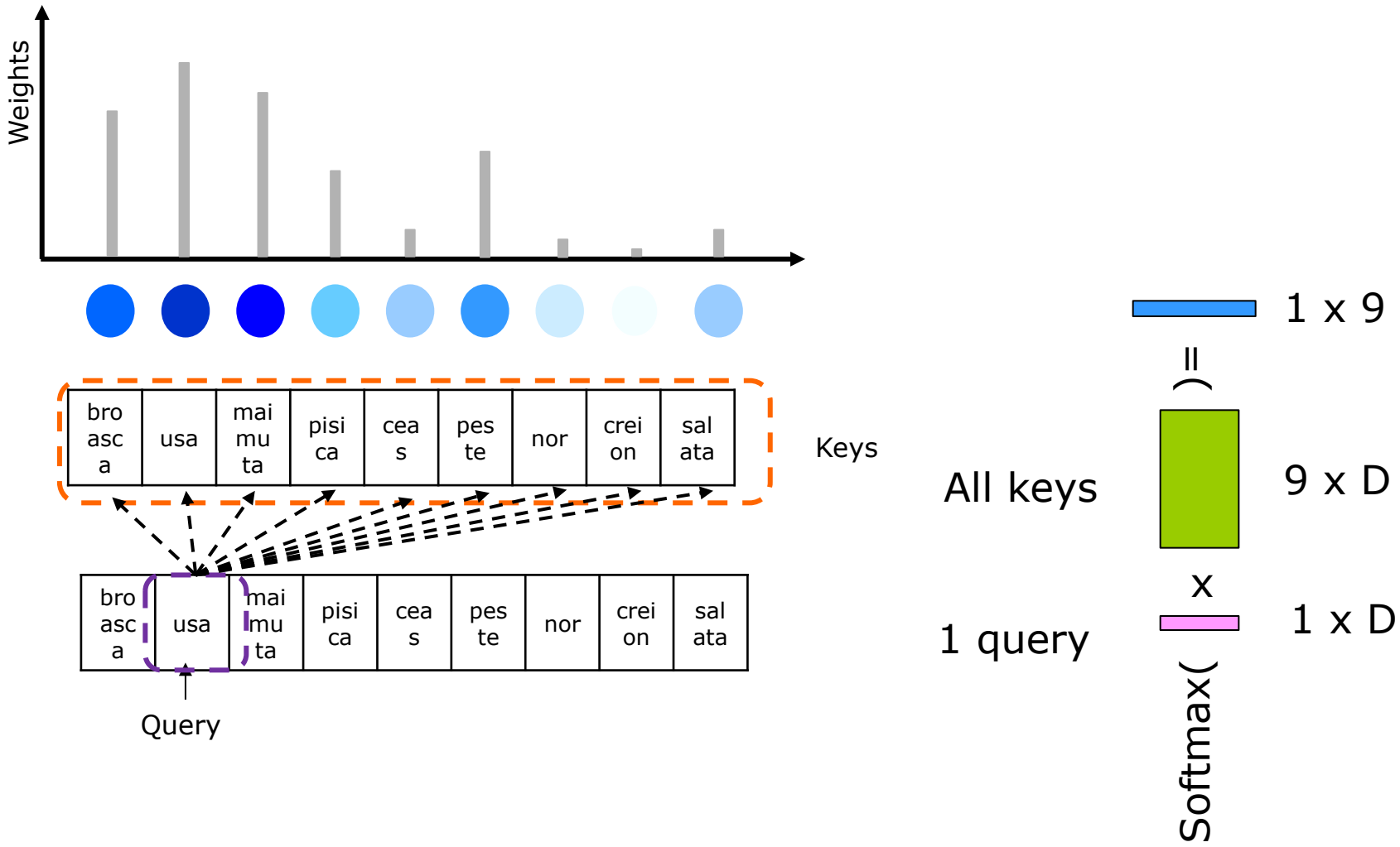
Mecanismul de “atentie” (attention)



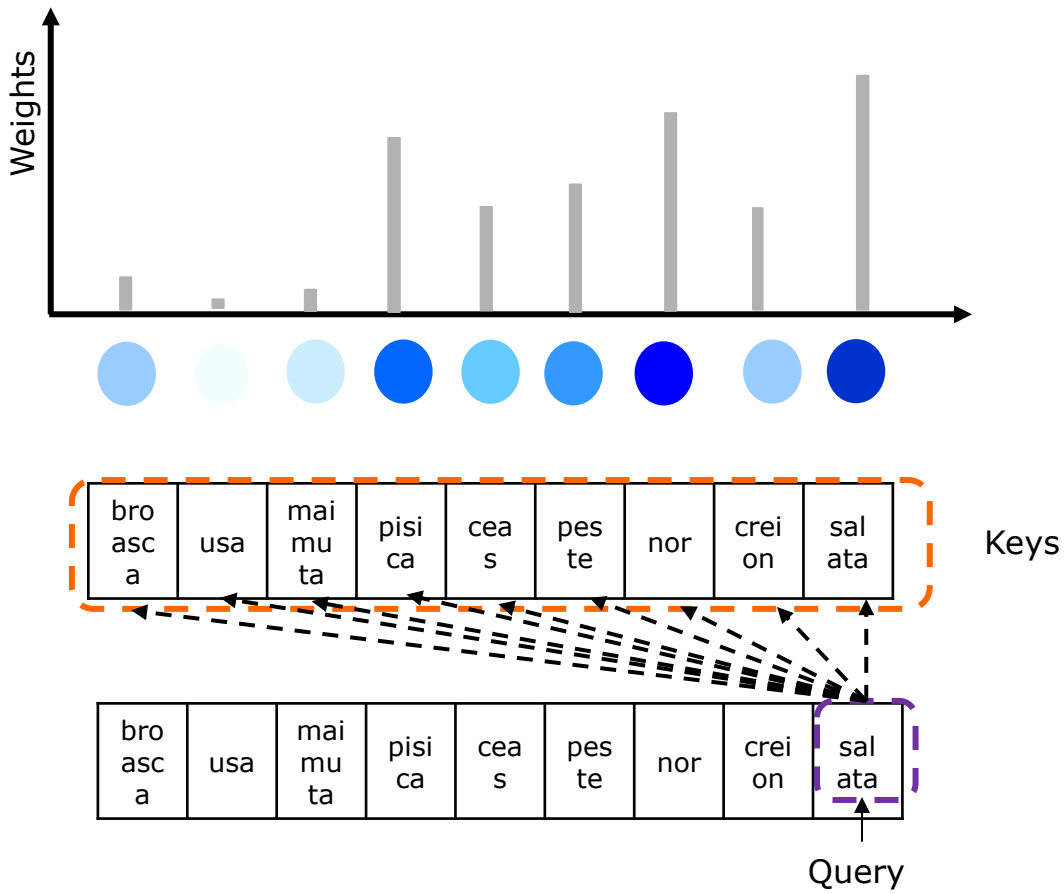
Mecanismul de “atentie” (attention)

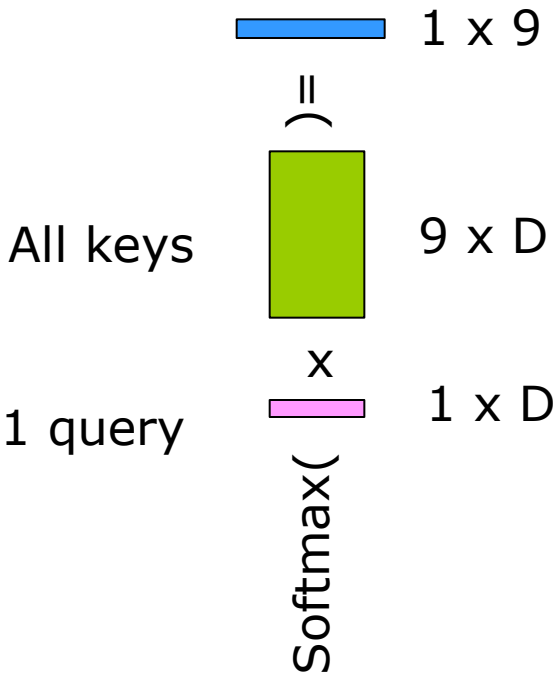


Mecanismul de “atentie” (attention)

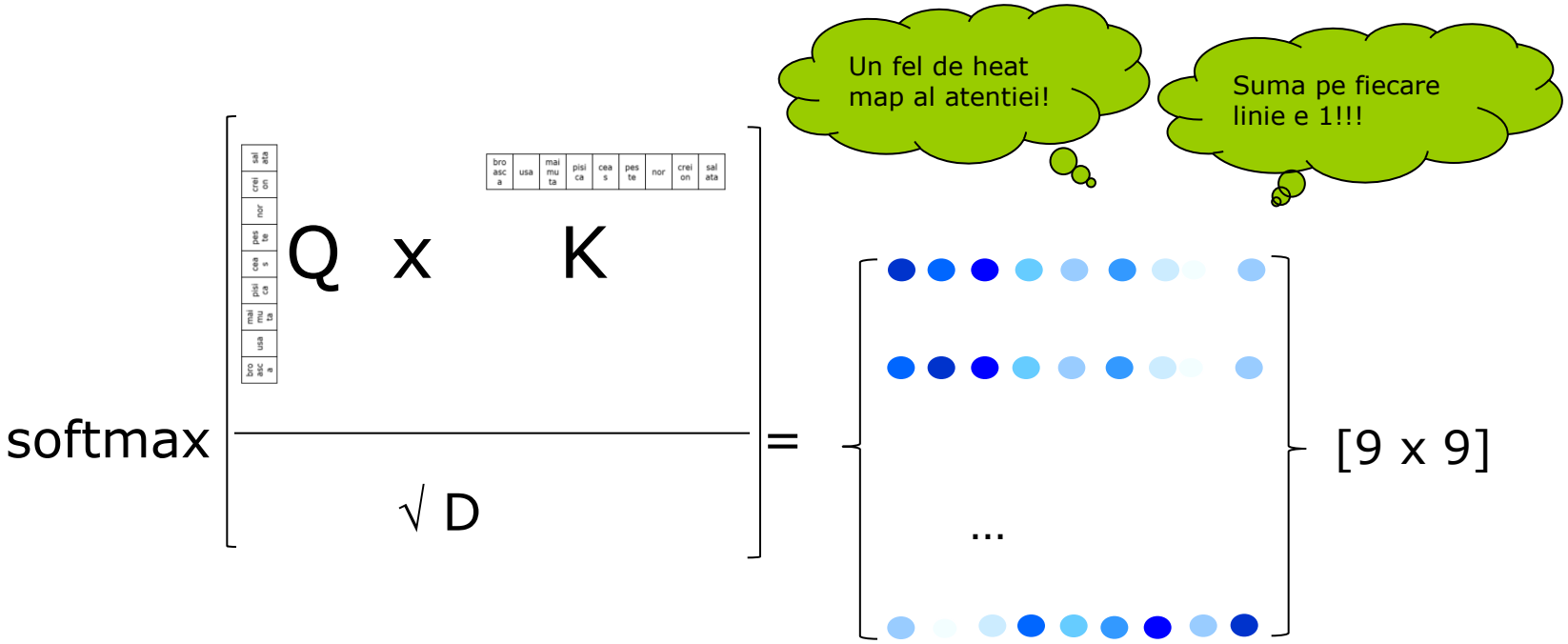


Mecanismul de “atentie” (attention)

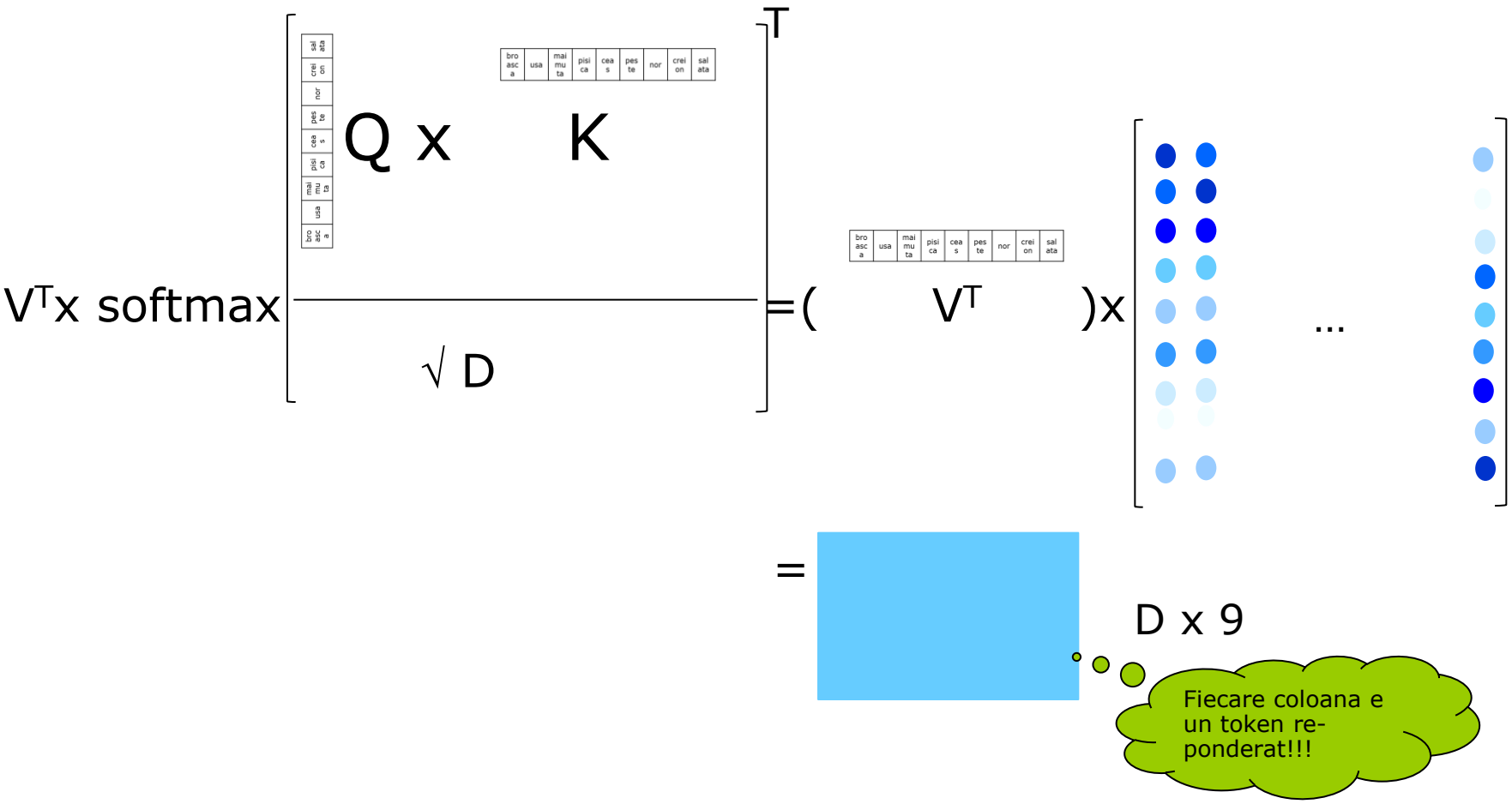




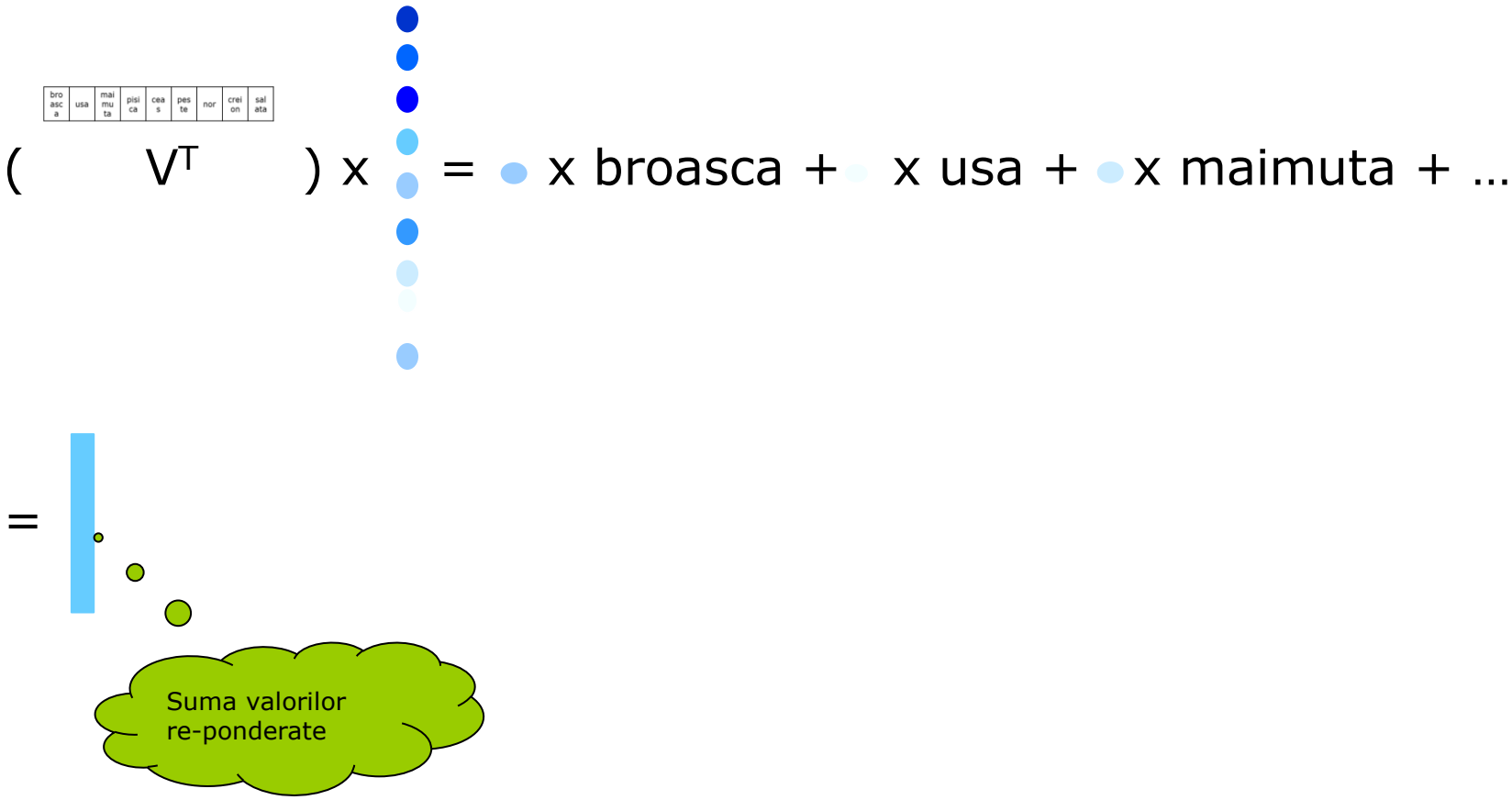
Mecanismul de “atentie” (attention)



Mecanismul de “atentie” (attention)



Mecanismul de “atentie” (attention)



Mecanismul de “atentie” (attention)

- Input = embedding-uri (de lungime n) pt v cuvinte – matrice v x n
- V – matrice v x v v – nr de cuvinte
- K – matrice n x d n – nr de features (lungimea unui embedding)
- Q – matrice n x d d – nr de features abstracte

Features

x_1	x_2	x_3	x_4
2	0	0	2
0	1	0	0
0	2	1	0
0	0	1	1
2	0	0	0
1	0	1	1

Mecanismul de “atentie” (attention)

- Input = embedding-uri (de lungime n) pt v cuvinte – matrice v x n
- V – matrice v x v v – nr de cuvinte
- K – matrice n x d n – lungimea reprezentarii dense (embedding)
- Q – matrice n x d d – nr de features abstracte

Features

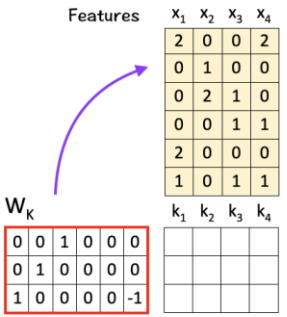
x_1	x_2	x_3	x_4
2	0	0	2
0	1	0	0
0	2	1	0
0	0	1	1
2	0	0	0
1	0	1	1

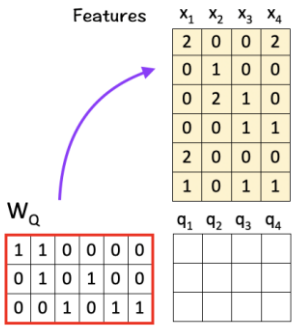
Transformare

input -> (input x K) x (Q^T x $input^T$) x (V x input) -> features

(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v) x (v,n))

(v,n) -> (v,d) x (d,v) x (v,n)





Mecanismul de “atentie” (attention)

- Input = embedding-uri (de lungime n) pt v cuvinte – matrice v x n
- V – matrice v x v v – nr de cuvinte
- K – matrice n x d n – nr de features (lungimea unui embedding)
- Q – matrice n x d d – nr de features abstracte

Features

	x_1	x_2	x_3	x_4
	2	0	0	2
	0	1	0	0
	0	2	1	0
	0	0	1	1
	2	0	0	0
	1	0	1	1

Transformare

input -> input x K x Q^T x $input^T$ x V x input -> features

(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v) x (v,n))

(v,n) -> (v,d) x (d,v) x (v,n)

(v,n) -> (v,v) x (v,n)

MatMul (K^TQ)

	q_1	q_2	q_3	q_4
	2	1	0	2
	0	1	1	1
	3	2	2	1

k_1^T	0	0	1
k_2^T	2	1	0
k_3^T	1	0	-1
k_4^T	0	0	1

Mecanismul de “atentie” (attention)

- Input = embedding-uri (de lungime n) pt v cuvinte – matrice v x n
- V – matrice v x v v – nr de cuvinte
- K – matrice n x d n – nr de features (lungimea unui embedding)
- Q – matrice n x d d – nr de features abstracte

Features

x_1	x_2	x_3	x_4
2	0	0	2
0	1	0	0
0	2	1	0
0	0	1	1
2	0	0	0
1	0	1	1

Transformare

input -> input x K x Q^T x $input^T$ x V x input -> features

(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v) x (v,n))

(v,n) -> (v,d) x (d,v) x (v,n)

(v,n) -> (v,v) x (v,n)

Scale

$$\frac{\square}{\sqrt{dk}} \approx \left\lfloor \frac{\square}{2} \right\rfloor$$

1	1	1	0
2	1	0	2
0	0	-1	0
1	1	1	0

Scalare

input -> input x K x Q^T x $input^T$ x V x input -> features

(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v) x (v,n))

(v,n) -> (v,d) x (d,v) x (v,n)

(v,n) -> (v,v) / **√d** x (v,n)

Normalizare

input -> input x K x Q^T x $input^T$ x V x input -> features

(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v) x (v,n))

(v,n) -> (v,d) x (d,v) x (v,n)

(v,n) -> **softmax (**(v,v) / **√d**) x (v,n)

Mecanismul de “atentie” (attention)

- Input = embedding-uri (de lungime n) pt v cuvinte – matrice v x n
- V – matrice v x v v – nr de cuvinte
- K – matrice n x d n – nr de features (lungimea unui embedding)
- Q – matrice n x d d – nr de features abstracte

Features

	x_1	x_2	x_3	x_4
	2	0	0	2
	0	1	0	0
	0	2	1	0
	0	0	1	1
	2	0	0	0
	1	0	1	1

Transformare

input -> input x K x Q^T x $input^T$ x V x input -> features

(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v

(v,n) -> (v,d) x (d,v) x

(v,n) -> (v,v) x

(v,n) -> (v,n)’

Scalare

input -> input x K x Q^T x $input^T$ x V x input -> fe

(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v

(v,n) -> (v,d) x (d,v) x

(v,n) -> (v,v) / **√v** x

(v,n) -> (v,n)’

W_V

10	0	0	0	0	0
0	0	0	10	0	0
0	10	0	0	0	0

v_1	v_2	v_3	v_4
20	0	0	20
0	0	10	10
0	10	0	0

MatMul

.3	.3	.4	.1
.9	.3	.2	.7
.1	.1	0	.1
.3	.3	.4	.1

Attention Weight Matrix (A)

z_1	z_2	z_3	z_4

Attention Weighted Features

Normalizare

input -> input x K x Q^T x $input^T$ x V x input -> features

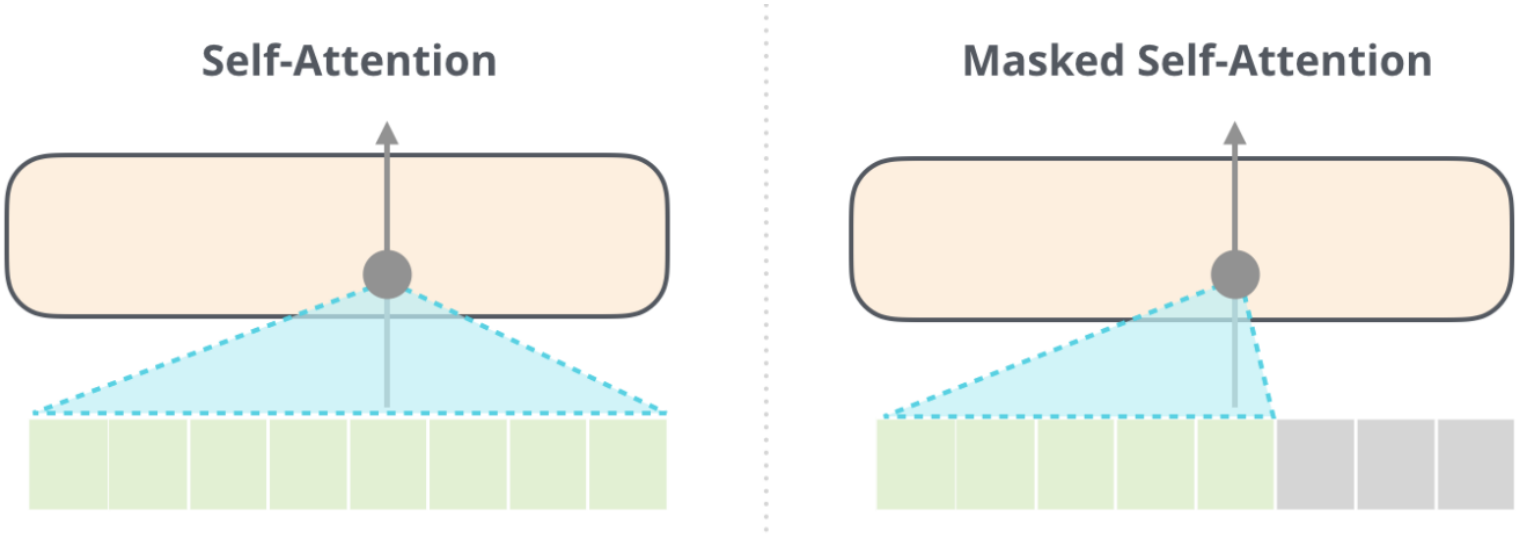
(v,n) -> ((v,n) x (n,d)) x ((d,n) x (n,v)) x ((v,v) x (v,n))

(v,n) -> (v,d) x (d,v) x (v,n)

(v,n) -> **softmax (**(v,v) / **√v)** x (v,n)

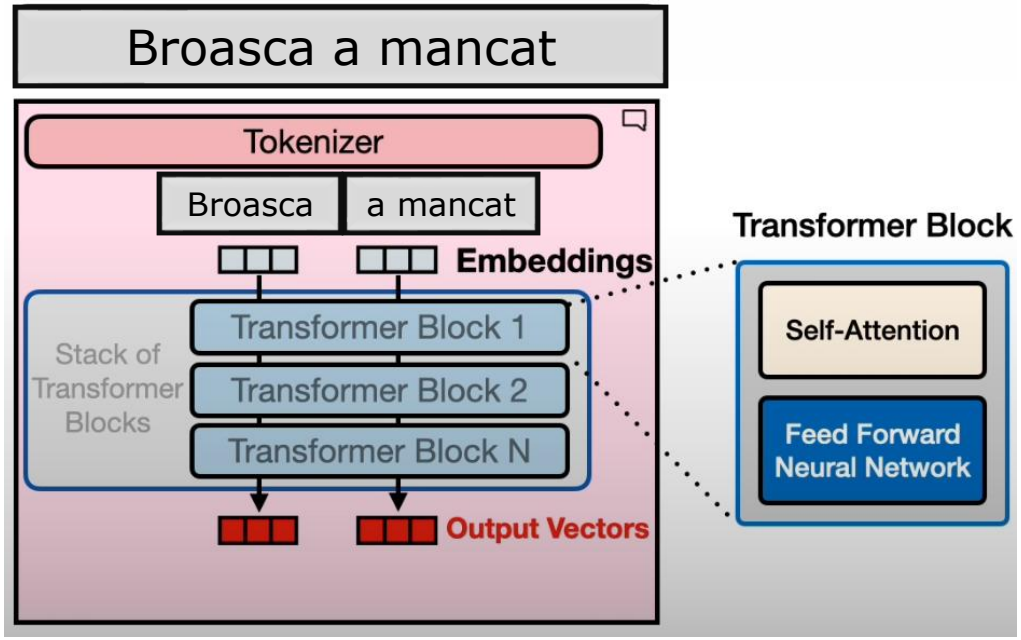
(v,n) -> (v,n)

Mecanismul de “atentie” (attention)



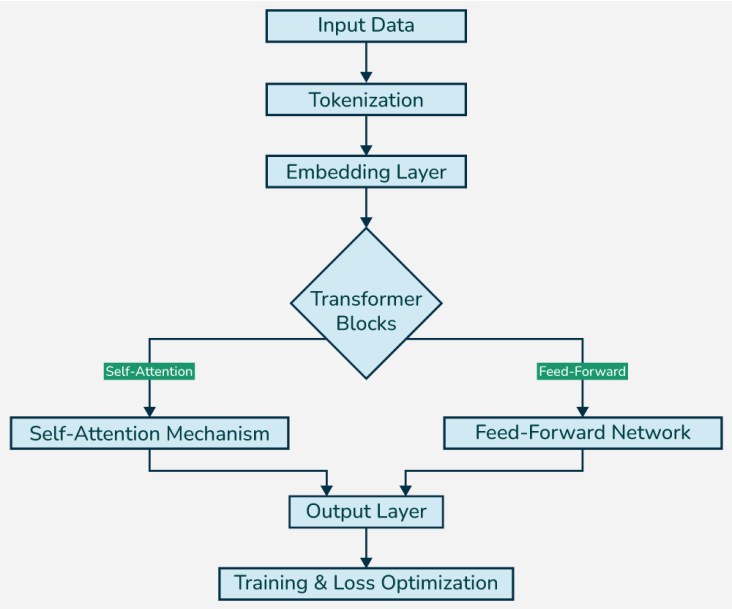
Procesarea textelor/limbajelor

- Large language Models (LLMs)
  - Tokenisation
  - Embeddings
  - Transformer & Self-attention mechanism
  - **Feed-forward network**



Procesarea textelor/limbajelor

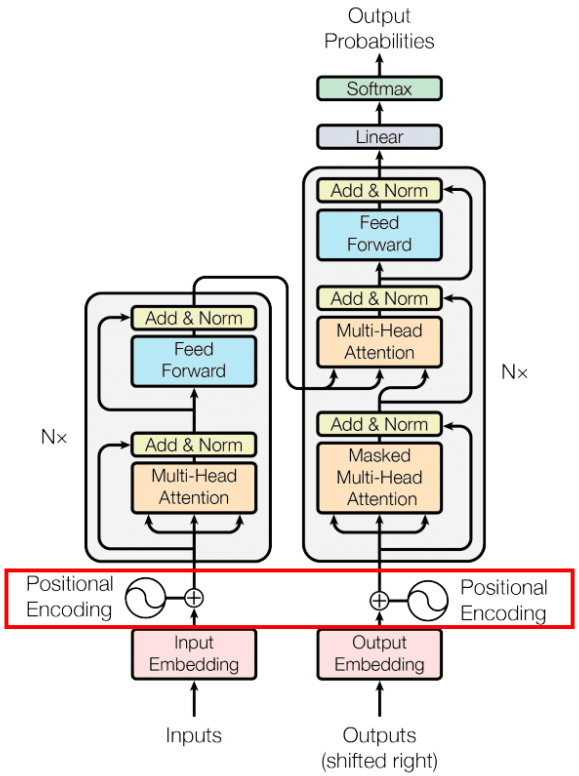
- Large language Models (LLMs)
  - Tokenisation
  - Embeddings
  - Transformer & Self-attention mechanism
  - **Feed-forward network**
    - **Normalisation layers**
    - **Softmax layers**
    - **Loss functions**



Procesarea textelor/limbajelor

- Large language Models (LLMs)
  - How large an LLM is?
    - |Vocabular| = V
      - Words/tokens
    - |Embedding| = E (model dimension)
    - |input sequence| = L (context window)
    - No of blocks NB
    - Size of MLP's hidden layers H

$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$




```
GPT2Model(
        TokenEmbedding,             → V x E param
        PositionalEMbedding,        → L x E param
        Dropout,                    → 0
        NB x TransformerBlock(
            Normalisation,          → 2 x E
            MSA,                    → 3 x (E x E + E) + E x E + E
            Normalisation,          → 2E
            MLP(
                Linear,             → E x H + H
                Linear,             → H X E + E
            )
        )
        Normalisation,              → 2E
        Linear                      → E X V
)
```

#param = param(embeddings) + param(transformer) + param(others)

= (WTextEmb + WPosEmb) + NB * (Norm + Attn + Norm + MLP) + Norm + Linear

= (V x E + L x E) + NB * (2E + (3 * (E x E + E) + (E x E + E)) + 2E + (E x H + H + H x E + E)) + 2E + E X V

= E x (V + L) + NB x ($4E^2$ + 9E + 2EH + H) + 2E + E x V

Procesarea textelor/limbajelor

- Large language Models (LLMs)
  - How large an LLM is?
    - |Vocabular| = V
      - Words/tokens
    - |Embedding| = E (model dimension)
    - |input sequence| = L (context window)

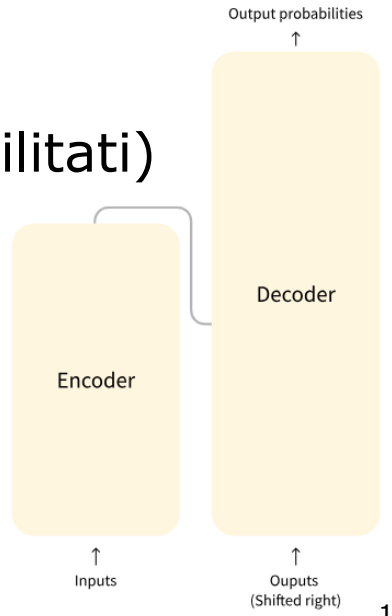
    - No of blocks NB
    - Size of MLP's hidden layers H

$$\#param = E \times (V + L) + NB \times (4E^2 + 9E + 2EH + H) + 2E + E \times V$$

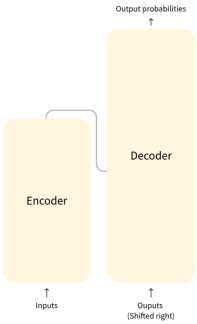
	V	E	L	NB	H	#param
GPT2	50 257	768	1 024	12	3 072	124 439 808
GPT3	50 257	12 288	2 048	96	12 288	87 627 632 640

Tranformers

- Architecture
  - Encoder
    - Primeste cuvinte
    - Construieste reprezentari (features) ale cuvintelor
  - Decoder
    - Primeste
      - reprezentari (features) ale cuvintelor
      - Alte inputuri
    - Genereaza secvente de cuvinte (ca probabilitati)



Tranformers

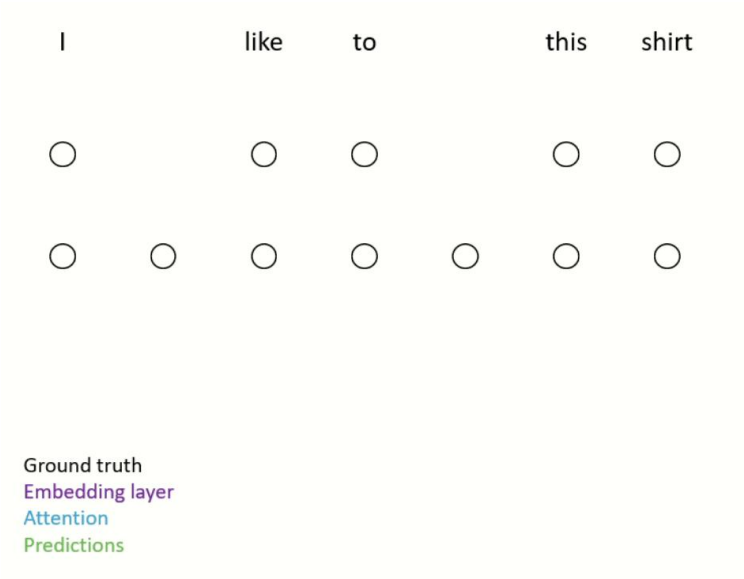


- Arhitectura
  - Modele bazate doar pe encoder
    - Proiectate pentru a invata sa prezica output-uri specifice unui task
      - In: texte
      - Out: labels sau tokens
    - Task-uri pe care le pot rezolva
      - Clasificare de propozitii
      - Identificarea in texte a unor sentimente/emotii
    - Ex. BERT, RoBERTa
  - Modele bazate doar pe decoder
    - Proiectate pentru a genera noi texte (decoding realizat in stil auto-regresiv = generare de secvente, *token by token*, fiecare token fiind conditionat de token-urile anterior generate)
      - In: embeddings
      - Out: texte
    - Task-uri pe care le pot rezolva
      - Raspuns la diferite queries
    - Ex. GPT
  - Modele bazate pe encoder-decoder
    - Proiectate pentru a genera noi texte
      - In: texte
      - Out: texte
    - Task-uri pe care le pot rezolva
      - rezumat de texte
      - Traducere de texte
    - Ex. BART, T5

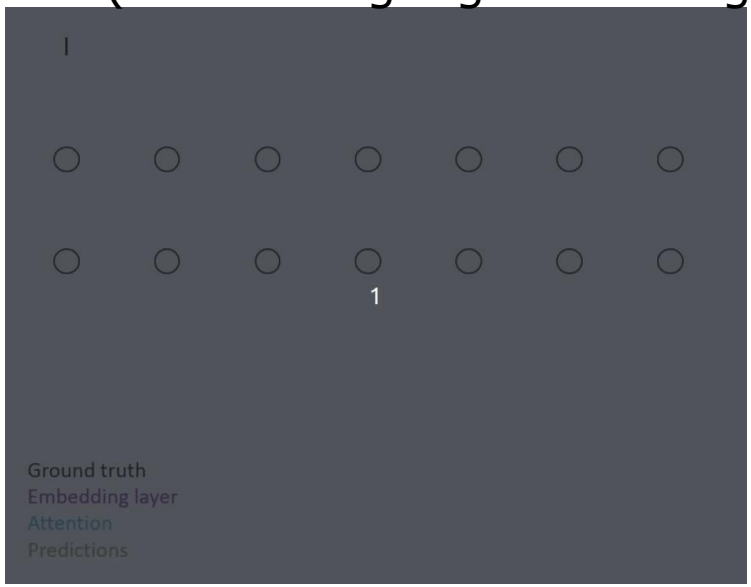
Tranformers

- Architecture
- Pre-training – unsupervised

- Masked language modelling

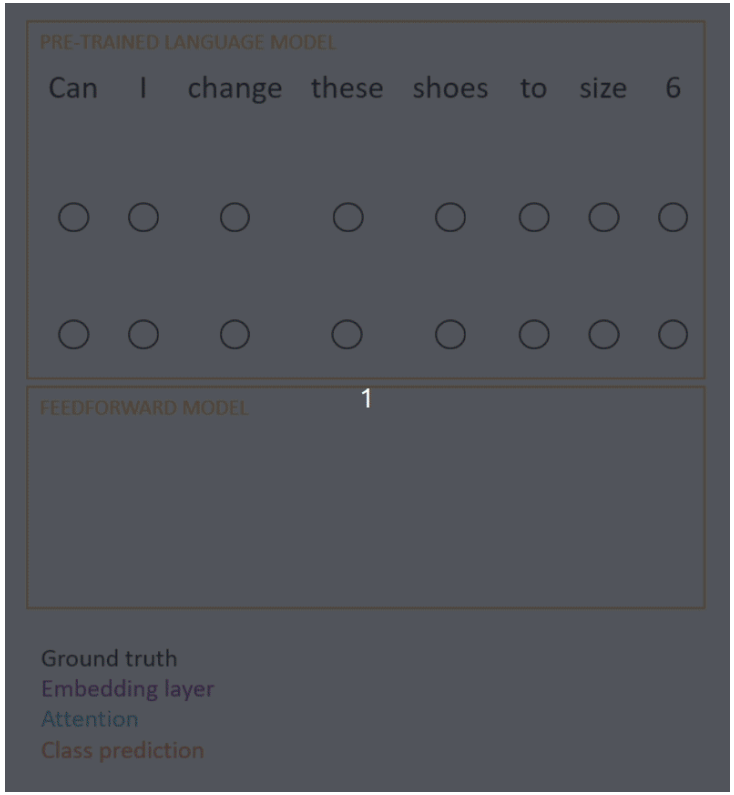


- Next word prediction (causal language modeling)



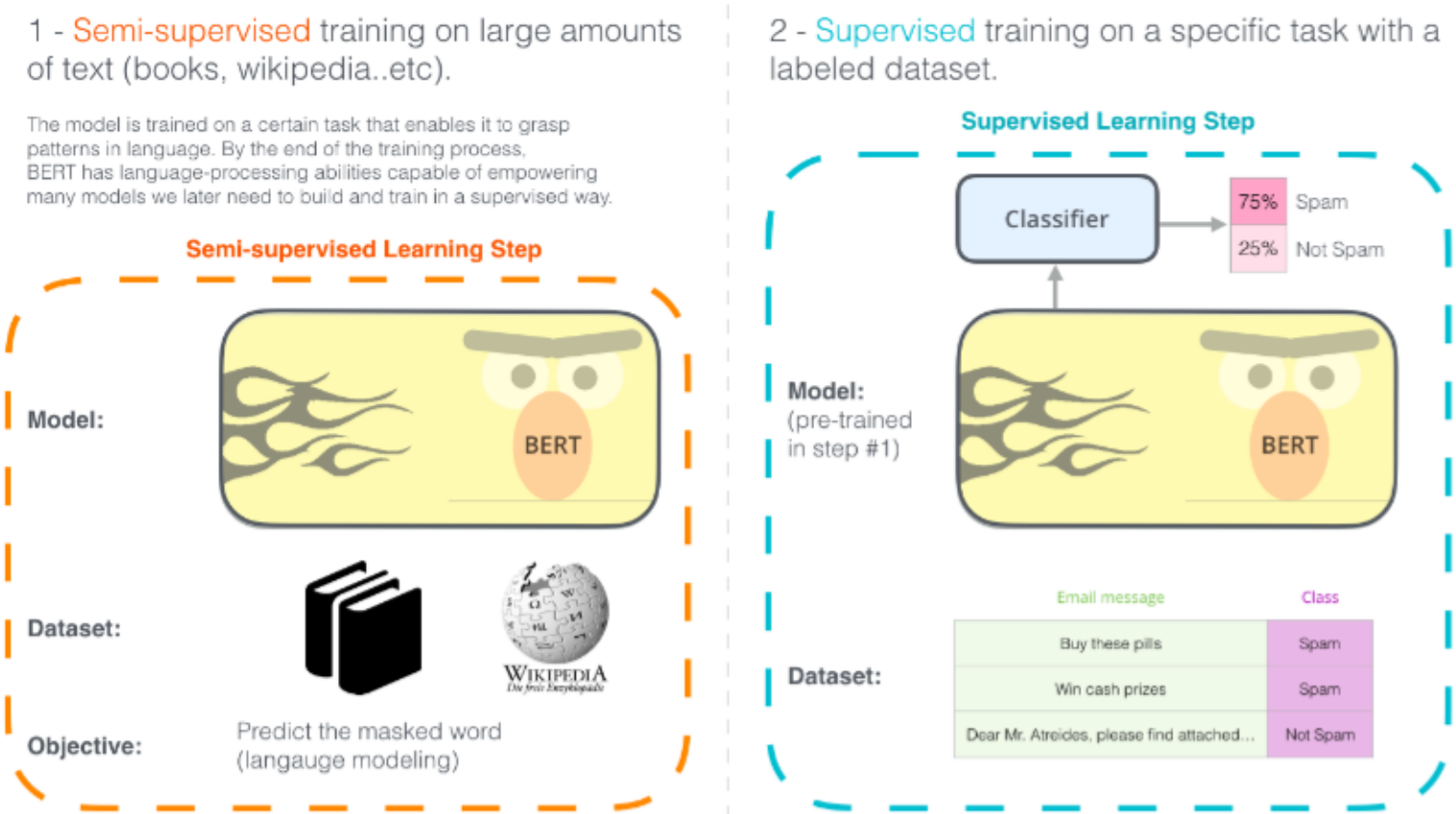
Tranformers

- Architecture
- Pre-training – unsupervised
- Fine-tunning
  - Specific task
    - Another model -> Additional layers (params)



Tranformers

- Fine-tunning
  - Specific task
    - Another model -> Additional layers (params)



Transformers

- Fine-tunning
  - Specific task
    - Email classification / sentiment analysis of product reviews
  - Model
    - BERT + a simple classifier (with softmax)
    - BERT = **B**idirectional **E**ncoder **R**epresentations from **T**ransformers
      - Versions:
        - base
          - 12 encoder layers = transformer blocks
          - 768 hidden units in the FFN (feed forward network)
          - 12 attention heads
        - Large
          - 24 encoder layers
          - 1024 hidden units in the FFN
          - 16 attention heads

Original transformer
- 6 encoder layers
- 512 hidden units
- 8 attention heads

Generative AI

- Prompt engineering
  - The practice of designing and optimising text input to generative AI models to obtain desired results
  - “art of communicating” with GenAI models
    - Explain transformers.
    - Explain the architecture of transformers.
    - Explain the architecture of transformers for text processing.

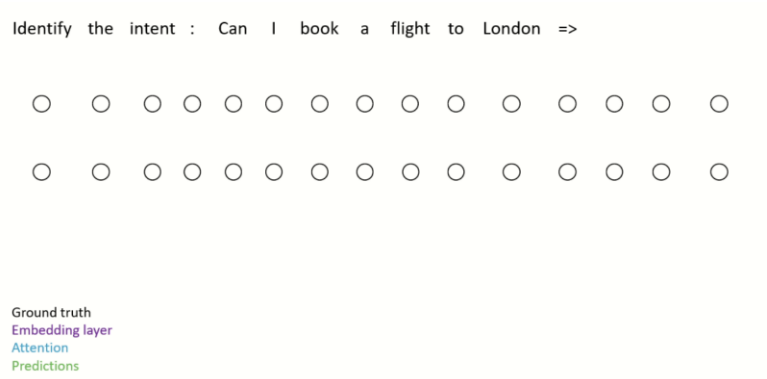
LLMs - prompts

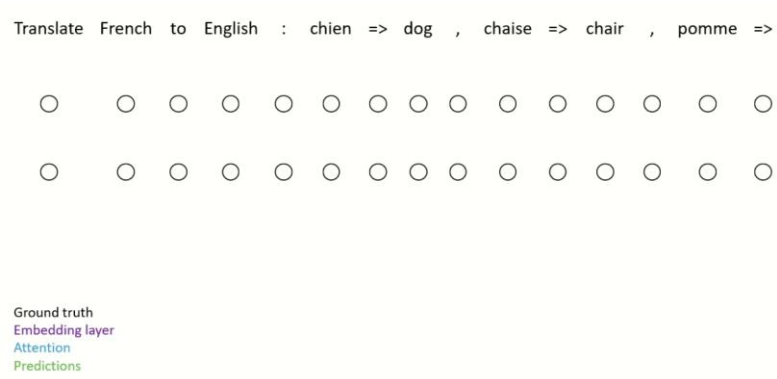
- Prompt = the input provided to the AI model to elicit a specific response
  - Questions
  - Keywords for complex instructions
  - Code snippets

- Prompt components
  - Task (desired goal) and instructions
    - the text in the prompt that you want the model to provide a response for
    - E.g.
      - *What are the colors in the rainbow?*
      - *Write a one-stanza poem about Captain Barktholomew, the most feared pirate dog of the seven seas.*
  - System instruction
    - instructions that get passed to the model before any user input in the prompt
  - Context
    - information included in the prompt that the model uses or references when generating a response
    - background knowledge to help the model understand you
  - Constraints
  - Examples
  - Reasoning steps
  - Response format

LLMs- prompt

- Types of prompts
  - Simple
  - With examples (k-shot learning)





LLMs- prompt

- Types of prompts
  - Direct prompts (zero-shot)
    - providing the model with a direct instruction or question without any additional context or examples
    - E.g. translation, summarization
  - One-, few- and multi-shot prompts
    - providing the model with one or more examples of the desired input-output pairs before presenting the actual prompt
    - E.g.
  - Chain of Thought Prompts
    - encourages the model to break down complex reasoning into a series of intermediate steps
  - Zero-shot CoT Prompts
    - asking the model to perform reasoning steps

Transformers - roadmap

- Resources
  - https://jalammar.github.io/illustrated-transformer/
  - Transformers (how LLMs work) explained visually | DL5 – YouTube https://www.youtube.com/watch?v=wjZofJX0v4M
  - LLM Visualization https://bbycroft.net/llm
  - nanoGPT https://www.youtube.com/watch?v=kCc8FmEb1nY
  - Attention https://lilianweng.github.io/posts/2018-06-24-attention/
  - https://arxiv.org/pdf/1706.03762
  - https://github.com/karpathy/nanoGPT
  - https://github.com/karpathy/ng-video-lecture
  - https://colab.research.google.com/drive/1JMLa53HDuA-i7ZBmqV7ZnA3c_fvtXnx-?usp=sharing
  - https://paperswithcode.com/paper/attention-is-all-you-need
  - https://www.youtube.com/watch?v=kCc8FmEb1nY

Recap

- Generative AI
  - Large language Models (LLMs)
    - Tokenisation
    - Embeddings
    - Transformer & Self-attention mechanism
    - Feed-forward network
  - Architectures
  - Fine-tuning
  - Prompt engineering

Cursul următor

A. Scurtă introducere în Inteligenţa Artificială (IA)

B. Sisteme inteligente

- **Sisteme care învaţă singure**
  - Arbori de decizie
  - Reţele neuronale artificiale – transformers
  - RAG systems
  - Maşini cu suport vectorial
  - Algoritmi evolutivi
- Sisteme bazate pe reguli
- Sisteme hibride

C. Rezolvarea problemelor prin căutare

- Definirea problemelor de căutare
- Strategii de căutare
  - Strategii de căutare neinformate
  - Strategii de căutare informate
  - Strategii de căutare locale (Hill Climbing, Simulated Annealing, Tabu Search, Algoritmi evolutivi, PSO, ACO)
  - Strategii de căutare adversială

- Informaţiile prezentate au fost colectate din diferite surse de pe internet, precum şi din cursurile de inteligenţă artificială ţinute în anii anteriori de către:
  - Conf. Dr. Mihai Oltean – www.cs.ubbcluj.ro/~moltean
  - Lect. Dr. Crina Groşan - www.cs.ubbcluj.ro/~cgrosan
  - Prof. Dr. Horia F. Pop - www.cs.ubbcluj.ro/~hfpop
  - Prof. Dr. Radu Ionescu - https://raduionescu.herokuapp.com/